AI-BASED LOCATION-ADAPTIVE LANDSLIDE EARLY-WARNING USING ABNORMAL ENVIROMENTAL CHANGE AND TEMPORAL RISK ESCALATION

Project Report - I

Submitted by

N.AATHI LAKSHMI (95192309001)

F.NAVYA FRANCIS (95192309034)

R.RAGAVI (95192309037)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

P.S.R. ENGINEERING COLLEGE, SIVAKASI

(An Autonomous Institution, Affiliated to Anna University, Chennai)

ANNA UNIVERSITY : CHENNAI 600 025

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled **"AI-Based Location-Adaptive Landslide Early-Warning Using Abnormal Environmental Change and Temporal Risk Escalation"** is the bonafide work of **"N. Aathi Lakshmi (95192309001), F. Navya Francis (95192309034), R. Ragavi (95192309037)"** who carried out the project work under my supervision.

SIGNATURE
Ms. J. JERALDIN AMALA PRIYA M.E.,
SUPERVISOR
ASSISTANT PROFESSOR
Department of AI & DS
P.S.R. Engineering College,
Sivakasi - 626140.

SIGNATURE
Dr C. NATARAJAN M.E., Ph.D.,
HEAD OF THE DEPARTMENT
ASSOCIATE PROFESSOR
Department of AI & DS
P.S.R. Engineering College,
Sivakasi - 626140.

Submitted for the PROJECT-I Viva-Voce Examination to be held on……..

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We take this opportunity to thank all those who helped towards the successful completion of this project. At the very outset, we thank the Almighty for His profuse blessings showered on us. We thank our beloved parents whose encouragement and support helped us to complete our project successfully.

It is our greatest pleasure to express our sincere thanks to **Thiru R. SOLAISAMY,** Correspondent and Director **Er. S. VIGNESWARI ARUNKUMAR B.Tech.,** P.S.R. Engineering College, Sivakasi, for providing the required facilities and suitable infrastructure to carry out this project.

It is our privilege to convey our sincere thanks to our respected honorable Principal **Dr. J. S. SENTHIL KUMAAR, M.E., Ph.D.,** for giving us permission to do this project in our institution.

We wish to extend and express our sincere thanks to our adored Head of the Department **Dr. C. NATARAJAN, M.E., Ph.D., Associate Professor** for his excellent guidance and motivation to complete our project proficiently.

We also wish to express our hearty thanks to our Project Guide, **Miss. J. JERALDIN AMALA PRIYA, M.E., Assistant Professor** for his excellent guidance and constant encouragement during this project work.

We also wish to express our sincere thanks to our Esteemed Project Coordinators **Dr. C. NATARAJAN, M.E., Ph.D., Associate Professor And Miss. G. SEENIAMMAL, M.E., Assistant Professor** for their continuous guidance and valuable suggestions, without whom this project would not have become a reality.

We are also bound to thank all the Teaching and Non-Teaching staff members of our **Artificial Intelligence and Data Science Department** who supported us with their wishes and prayers that brought success and completion to our project.

ABSTRACT

The Landslides are hazardous natural events influenced by environmental conditions such as rainfall, soil moisture and terrain characteristics. These conditions vary across locations and over time, making static maps and fixed thresholds less effective for continuous risk assessment. This project proposes an AI-Based Location-Adaptive Landslide Early-Warning System that combines machine learning with temporal environmental analysis. Historical landslide and environmental data are processed to generate temporal features and predict probability-based landslide risk. The system uses location-specific historical baselines to identify abnormal environmental changes and analyses risk trends to determine whether conditions are increasing, stable or decreasing. Based on these factors, progressive warning states are generated as NORMAL, WATCH, WARNING and HIGH RISK. The system also provides explainable alerts, event-based evaluation and lead-time analysis. A what-if simulation module evaluates how changes in rainfall and soil moisture affect predicted risk and warning states. The complete framework is integrated into a web dashboard using React, TypeScript and FastAPI for visualizing locations, warnings, risk trends, evaluation results and simulation outputs. The proposed framework supports location-adaptive, temporal and explainable landslide early-warning analysis.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iv
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	xi
1	**INTRODUCTION**	1
	1.1 INTRODUCTION	1
	1.2 BACKGROUND	2
	1.3 PROBLEM STATEMENT	4
	1.4 MOTIVATION	5
	1.5 OBJECTIVES	7
	1.6 SCOPE	8
	1.7 APPLICATIONS	11
2	**LITERATURE REVIEW**	12
	2.1 INTRODUCTION	12
	2.2 TERRITORIAL EARLY-WARNING SYSTEMS	12
	2.3 RAINFALL THRESHOLD METHODS	13
	2.4 MACHINE-LEARNING BASED LANDSLIDE ANALYSIS	14
	2.5 MONITORING AND LOCATION-ADAPTIVE ANALYSIS	15
	2.6 PERFORMANCE EVALUATION OF EARLY-WARNING SYSTEMS	15
	2.7 LITERATURE SURVEY	17
	2.8 RESEARCH GAP	17

3	**SYSTEM ARCHITECTURE**	18
	3.1 EXISTING SYSTEM	18
	3.2 PROPOSED SYSTEM	19
4	**SYSTEM SPECIFICATION**	21
	4.1 HARDWARE REQUIREMENTS	21
	4.2 SOFTWARE REQUIREMENTS	22
	4.3 DATASET SPECIFICATION	23
	4.4 ENVIRONMENTAL DATA SPECIFICATION	24
	4.5 DEVELOPMENT ENVIRONMENT	25
5	**SYSTEM IMPLEMENTATION**	26
	5.1 SYSTEM OVERVIEW	26
	5.2 SYSTEM DESIGN	27
	5.3 MODULE 1 – PROJECT SETUP	28
	5.4 MODULE 2 – DATA PIPELINE	29
	5.5 MODULE 3 – AI RISK MODEL	30
	5.6 MODULE 4 – LOCATION-ADAPTIVE ANALYSIS	32
	5.7 MODULE 5 – EARLY-WARNING ENGINE	34
	5.8 MODULE 6 – EVENT AND LEAD-TIME EVALUATION	36
	5.9 MODULE 7 – WHAT-IF SIMULATION	37
	5.10 MODULE 8 – WEB DASHBOARD	39
	5.11 TECHNOLOGY STACK	41

6	**RESULTS AND DISCUSSION**	43
	6.1 DATASET PROCESSING RESULTS	43
	6.2 TEMPORAL FEATURE RESULTS	44
	6.3 RISK PREDICTION RESULTS	45
	6.4 LOCATION-ADAPTIVE RESULTS	46
	6.5 EARLY-WARNING RESULTS	47
	6.6 EVENT EVALUATION RESULTS	50
	6.7 WHAT-IF SIMULATION RESULTS	51
	6.8 DASHBOARD RESULTS	55
7	**TESTING AND VALIDATION**	57
	7.1 TESTING METHODOLOGY	57
	7.2 UNIT TESTING	58
	7.3 INTEGRATION TESTING	58
	7.4 API TESTING	59
	7.5 FRONTEND TESTING	60
	7.6 FINAL VALIDATION	61
8	**CONCLUSION AND FUTURE WORK**	63
	8.1 CONCLUSION	63
	8.2 LIMITATIONS	63
	8.3 FUTURE WORK	64
	REFERENCES	65
	APPENDIX	68

LIST OF TABLES

TABLE NO.	TITLE	PAGE No.
2.1	LITERATURE SURVEY	17
4.1	HARDWARE REQUIREMENTS	21
4.2	SOFTWARE REQUIREMENTS	22
4.3	DATASET DESCRIPTION	24
5.2	WARNING STATE CONFIGURATION	35
6.1	DATASET PROCESSING RESULTS	43
6.2	WARNING STATE DISTRIBUTION	47
6.3	WHAT-IF SIMULATION RESULTS	52
7.1	PHASE-WISE TESTING RESULTS	61
7.2	FINAL VALIDATION SUMMARY	62

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	OVERALL ARCHITECTURE OF THE PROPOSED SYSTEM	20
5.1	PROJECT DATA PROCESSING WORKFLOW	28
5.2	TEMPORAL FEATURE GENERATION WORKFLOW	30
5.3	AI RISK PREDICTION WORKFLOW	32
5.4	LOCATION-ADAPTIVE BASELINE AND ABNORMALITY DETECTION	33
5.5	EARLY-WARNING ENGINE WORKFLOW	36
5.6	EVENT AND LEAD-TIME EVALUATION WORKFLOW	37
5.7	WHAT-IF SIMULATION WORKFLOW	39
5.8	WEB DASHBOARD ARCHITECTURE	41
6.1	DATASET VALIDATION OUTPUT	44
6.2	TEMPORAL FEATURE OUTPUT	45
6.3	AI RISK PREDICTION OUTPUT	46
6.4	LOCATION-ADAPTIVE ANALYSIS OUTPUT	47
6.5	WARNING STATE DISTRIBUTION	48
6.6	RISK TRAJECTORY	48
6.7	ANOMALY VERSUS RISK	49
6.8	WARNING TRANSITIONS	49
6.9	LEAD-TIME DISTRIBUTION	50
6.10	DETECTION VERSUS MISSED EVENTS	51
6.11	SCENARIO RISK COMPARISON	53
6.12	RAINFALL SENSITIVITY CURVE	53

6.13	SOIL-MOISTURE SENSITIVITY CURVE	54
6.14	WARNING-STATE SCENARIO TRANSITION	54
6.15	FINAL WEB DASHBOARD	55
6.16	LOCATION DETAILS DASHBOARD	56
7.1	FRONTEND PRODUCTION BUILD / DASHBOARD VALIDATION	60

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	ARTIFICIAL INTELLIGENCE
ML	MACHINE LEARNING
EWS	EARLY-WARNING SYSTEM
GIS	GEOGRAPHIC INFORMATION SYSTEM
GLC	GLOBAL LANDSLIDE CATALOG
API	APPLICATION PROGRAMMING INTERFACE
CSV	COMMA-SEPARATED VALUES
REST	REPRESENTATIONAL STATE TRANSFER
ROC	RECEIVER OPERATING CHARACTERISTIC
AUC	AREA UNDER THE CURVE
F1	F1 SCORE
TP	TRUE POSITIVE
TN	TRUE NEGATIVE
FP	FALSE POSITIVE
FN	FALSE NEGATIVE

CHAPTER 1
INTRODUCTION

1.1 INTODUCTION

Landslides are one of the major natural hazards affecting mountainous and hilly regions around the world. A landslide occurs when soil, rock or debris moves downward along a slope due to gravitational forces. The occurrence of landslides can be influenced by rainfall, soil moisture, geological conditions, slope characteristics, land cover and other environmental factors[4]. Among these factors, rainfall is one of the important triggers of rainfall-induced landslides because prolonged or intense precipitation can increase soil moisture and reduce slope stability [3]. The behaviour of landslide-prone locations is not identical. Environmental conditions that are considered normal for one geographical location may be unusual for another location. Seasonal variations also influence the normal environmental state of a region. Consequently, applying one fixed threshold to all locations may not accurately represent the actual environmental conditions associated with landslide risk. Traditional susceptibility mapping provides valuable information about areas that are potentially susceptible to landslides[7]. However, susceptibility mainly describes the spatial likelihood of occurrence and does not necessarily indicate how the risk is changing at a particular time. Early-warning systems require temporal information because environmental conditions may change before a landslide event. Machine learning provides a useful approach for analysing multiple environmental variables and estimating landslide risk from historical observations. However, a single machine-learning probability is not sufficient to describe whether risk is increasing, decreasing or remaining persistent. A practical warning framework therefore needs additional temporal and location-specific analysis[20]. The proposed project addresses this requirement by integrating AI-based risk estimation with multi-temporal environmental analysis, location-specific historical baselines, abnormality detection, temporal risk escalation and event-oriented evaluation. The resulting system is designed as a complete research framework rather than only a machine-learning classifier[19].

1.2 BACKGROUND

Landslide early-warning systems have traditionally focused on rainfall thresholds, monitoring observations, and predefined warning criteria. Rainfall intensity and duration are commonly used because intense or prolonged rainfall can increase pore-water pressure and reduce slope stability [1], [3], [5]. However, rainfall alone does not always provide sufficient information to describe the changing condition of a slope. Antecedent environmental conditions, particularly soil moisture, can influence how a slope responds to subsequent rainfall [2], [6]. Therefore, incorporating multiple environmental variables can provide a broader representation of the conditions preceding a landslide.

Previous studies have also demonstrated the importance of monitoring strategies in landslide early-warning systems. Local monitoring can provide information about environmental and slope conditions at specific locations, while regional systems can provide wider spatial coverage [4], [20]. Satellite-derived rainfall and other remotely sensed observations have further increased the possibility of monitoring landslide-prone regions where ground-based observations are limited [7]. Global landslide inventories also provide historical information that can be used to investigate the spatial and temporal characteristics of landslide occurrence [8], [9].

Machine learning methods have increasingly been applied to landslide susceptibility and hazard assessment. These methods can identify relationships between landslide occurrence and multiple environmental or geographical factors [15], [16], [17]. Machine learning-based approaches can consider nonlinear relationships that may be difficult to represent using simple threshold-based methods. Recent research has also explored machine learning models for rainfall-induced landslide prediction and susceptibility assessment [10], [11]. Such approaches provide an opportunity to estimate a probability or risk value rather than relying only on a fixed threshold.

Rainfall-based warning methods remain important because rainfall is one of the major triggering factors for many landslides. Several studies have investigated rainfall intensity-duration relationships and rainfall thresholds for identifying conditions associated with possible landslide occurrence [11], [12], [13], [14]. This supports the need for approaches that consider the specific characteristics of individual locations rather than applying one identical threshold to every region.

Reanalysis datasets such as ERA5 provide historical environmental information that can support the analysis of rainfall, soil moisture and other atmospheric variables [21]. Such datasets allow environmental conditions to be analysed consistently across different locations and time periods. Soil-moisture information can also be used to represent the antecedent wetness of a region and improve the interpretation of rainfall-related landslide conditions [2], [22]. Satellite precipitation products such as IMERG provide another source of rainfall information that can support large-scale environmental monitoring [23].

The development of interpretable machine learning methods is also relevant to environmental risk prediction. A risk prediction is more useful for decision support when the factors contributing to the prediction can be examined rather than presenting only a numerical probability. Explainable machine learning approaches can therefore help users understand the relationship between environmental conditions and predicted risk [24], [25]. In the proposed system, this concept is extended through location-specific environmental analysis and explanation of the factors associated with changes in warning state.

Another important aspect of landslide early warning is the evaluation of warning performance. A warning system should not be assessed only by the number of predictions generated by the model. Detection of historical events, missed events, warning precision and the time available between a warning and an observed event provide additional information about system behaviour [5]. Performance assessment standards for territorial landslide early-warning systems have therefore emphasized systematic evaluation of warning performance [6].

Based on these considerations, the proposed system combines historical landslide information, environmental observations, temporal features, machine learning-based risk estimation, location-specific environmental baselines and temporal risk analysis. The change in risk over time is also considered before generating a warning state. This results in a framework that moves from simple susceptibility estimation toward a more dynamic representation of changing environmental risk. The overall objective is to provide an integrated research framework for analysing landslide risk and generating location-adaptive early-warning information using historical and environmental data [1], [5], [21].

1.3 PROBLEM STATEMENT

Existing landslide assessment approaches may provide susceptibility information, rainfall thresholds or machine-learning predictions, but a complete temporal early-warning framework requires additional components. A fixed threshold may not represent local environmental behaviour, while an isolated probability does not explain the progression of risk[9]. Therefore, the problem addressed in this project is the development of an integrated system that can estimate landslide risk using AI, compare environmental conditions with location-specific historical behaviour, identify abnormal changes, track temporal risk escalation and convert these observations into understandable warning states[13].

Rainfall-based thresholds are widely used for landslide early-warning systems because rainfall is an important triggering factor [1], [3], [5]. However, the relationship between rainfall and landslide occurrence can vary according to geographical and environmental conditions [11], [12]. Previous studies have also shown that antecedent soil moisture can influence the effectiveness of rainfall-based warning approaches [2]. Therefore, considering rainfall together with soil-moisture conditions can provide additional information about the environmental state of a location.

Machine-learning approaches provide another method for analysing multiple factors associated with landslide occurrence [10], [15]. These methods can identify complex relationships between environmental variables and landslide susceptibility. However, a machine-learning probability by itself does not necessarily describe whether the current environmental condition is unusual compared with the normal behaviour of a particular location. This creates a need for additional location-adaptive analysis [15], [16].

Historical environmental observations can be used to establish a reference representation of local environmental behaviour. Reanalysis datasets such as ERA5 provide consistent historical environmental information that can support this type of analysis [21]. Soil-moisture and rainfall information can also be examined across different time periods to identify changes preceding landslide events [2], [22]. Satellite-based rainfall products provide further opportunities for spatial environmental monitoring [7], [23].

Temporal analysis is particularly important because landslide risk can change as environmental conditions evolve. Previous rainfall, accumulated rainfall and changes in soil moisture may represent the environmental history of a location rather than only its current condition [2], [13]. Tracking risk values across time can therefore help identify whether the predicted risk is stable, increasing or decreasing.

The system also requires a mechanism to convert continuous risk information into understandable warning states. Territorial early-warning research emphasizes the importance of warning models and systematic performance evaluation [1], [5]. A multi-level warning structure can represent different stages of risk and provide a more understandable interpretation than a single numerical probability.

Finally, warning performance should be evaluated using historical landslide events rather than relying only on model-level metrics. Detection rate, missed events, warning precision and lead time can provide information about how the warning framework behaves in relation to observed events [6]. Explainable machine-learning approaches can further support interpretation by helping users understand the factors associated with individual predictions [24], [25].

Therefore, the proposed problem is addressed through an integrated framework combining AI-based risk prediction, location-specific baseline analysis, abnormal environmental change detection, temporal risk tracking and multi-level warning generation. The framework is intended to support research-oriented landslide risk assessment by connecting environmental observations with interpretable warning information [1], [5], [21].

1.4 MOTIVATION

The motivation for this project comes from the need to move from static landslide assessment toward temporally informed risk monitoring. A location may remain stable under normal environmental conditions and become more concerning when rainfall and soil moisture change significantly from its historical behaviour[9]. The project is also motivated by the need for understandable warnings. Disaster-management personnel and other users may find a simple numerical probability difficult to interpret without additional context[20-22]. Progressive warning states and explanations can provide a clearer representation of the current

model output. Another motivation is the evaluation of warning usefulness. A model may perform well on conventional classification metrics while still producing warnings that are too late or too frequent. Event-based evaluation therefore forms an important part of the proposed framework[19].

The motivation for this project comes from the need to move from static landslide assessment toward temporally informed risk monitoring. A location may remain stable under normal environmental conditions and become more concerning when rainfall and soil moisture change significantly from its historical behaviour[10]. The project is also motivated by the need for understandable warnings. Disaster-management personnel and other users may find a simple numerical probability difficult to interpret without additional context. Progressive warning states and explanations can provide a clearer representation of the current model output. Another motivation is the evaluation of warning usefulness[10]. A model may perform well on conventional classification metrics while still producing warnings that are too late or too frequent. Event-based evaluation therefore forms an important part of the proposed framework[17].

In addition, rainfall is widely recognized as an important triggering factor for rainfall-induced landslides, making precipitation-related variables valuable for dynamic risk assessment [1–5]. Previous studies have shown that antecedent rainfall, rainfall intensity, cumulative precipitation, and soil moisture conditions can influence slope stability [6–9]. However, environmental conditions vary significantly between geographical regions, and a model developed for one location may not perform equally well in another [10–12]. This highlights the importance of considering location-specific environmental characteristics while developing a landslide prediction system.

Recent advances in machine learning have provided opportunities to combine multiple environmental variables and identify complex relationships associated with landslide occurrence [13–16]. Machine-learning-based approaches can process historical rainfall, soil, terrain, and other environmental information to estimate the likelihood of landslide events. The increasing availability of geospatial and environmental data also makes it possible to develop systems that provide more frequently updated assessments rather than relying only on static susceptibility maps [17–19].

Another motivation is the practical gap between prediction models and end-user decision support. A technically accurate model may not be sufficient if its output cannot be easily understood or acted upon [20–22]. Therefore, the proposed system aims to transform model predictions into progressive risk categories such as low, moderate, high, and critical, allowing users to understand the severity of the current situation more easily. Location-based visualization can further help users identify areas of concern without requiring specialized knowledge of machine-learning models [23,24].

Finally, the project is motivated by the need to develop a practical and interpretable framework that can support early awareness and monitoring rather than attempting to replace professional disaster-management decisions. By combining historical patterns, current environmental conditions, machine-learning-based prediction, and understandable warning levels, the proposed system seeks to provide a more informative approach to landslide-risk monitoring. The framework can also be evaluated using both standard machine-learning metrics and event-oriented measures to understand whether warnings are timely, consistent, and useful for real-world monitoring [25].

1.5 OBJECTIVES

The primary objective of the project is to develop an AI-based location-adaptive landslide early-warning framework that combines machine-learning risk prediction with environmental anomaly detection and temporal risk escalation[12]. The project aims to process historical landslide events and environmental information, generate temporal features, estimate probability-based risk, construct location-specific historical baselines, identify abnormal environmental conditions and track changes in risk over time. The framework further aims to generate progressive warning states, provide explanations, evaluate warning performance using historical events and analyse environmental scenarios through what-if simulation[23].

The proposed framework also aims to integrate multiple environmental factors, particularly rainfall and soil-related conditions, to improve the representation of changing landslide risk [1–5]. Instead of relying only on fixed geographical susceptibility, the system is intended to consider how current environmental conditions differ from the normal historical behaviour of a particular location [6–9].

Temporal features such as recent rainfall, cumulative rainfall, rainfall intensity, antecedent rainfall and changes in environmental conditions can be incorporated to capture short-term variations in risk [10–12].

Another objective is to develop a location-adaptive baseline for each selected area. This baseline can represent the normal environmental behaviour of a location and provide a reference for identifying unusual conditions [13–15]. The system aims to detect environmental anomalies by comparing current observations with historical patterns and use these deviations as additional information for risk assessment. This approach can help distinguish ordinary environmental fluctuations from potentially concerning changes [16–18].

The project also aims to produce a probability-based risk score that can be transformed into understandable warning levels. Progressive warning states such as Low, Moderate, High and Critical can provide users with a clearer interpretation of changing risk rather than presenting only a numerical probability [19–21]. The framework is further intended to provide basic explanations for predictions by identifying the environmental factors that contribute to an increased or decreased risk level.

In addition, the project aims to evaluate the effectiveness of the warning framework using historical landslide events. Evaluation will consider conventional machine-learning measures as well as warning-specific aspects such as detection time, missed events, false warnings and warning duration [22–24]. Finally, a what-if simulation component is intended to allow users to explore hypothetical changes in environmental conditions and observe how the predicted risk may change. Through these objectives, the project seeks to develop an interpretable, location-aware and temporally responsive framework for landslide-risk monitoring and early warning [25].

1.6 SCOPE

The scope of the project covers the complete development of a modular landslide early-warning research system. It includes data validation, preprocessing, temporal feature engineering, machine-learning risk prediction, location-adaptive analysis, early-warning generation, event-based evaluation, scenario analysis and web-based visualization. The current implementation is primarily designed as a research and decision-support prototype using historical and processed environmental

information. The framework can be extended to operational applications when reliable real-time observations, forecast data and suitable validation procedures are available.

The data-processing component covers the collection, cleaning and preparation of historical landslide and environmental datasets. Relevant attributes such as geographical location, rainfall, soil moisture and other available environmental factors can be incorporated into the analysis. The preprocessing stage focuses on handling missing values, inconsistent records, duplicate observations and unsuitable data formats before model development.

The machine-learning component focuses on learning relationships between environmental conditions and historical landslide occurrences. Temporal features can be generated to represent recent and cumulative environmental changes, allowing the model to consider how conditions evolve over time. Different machine-learning algorithms can be evaluated based on their predictive performance and suitability for the available dataset.

The scope also includes location-adaptive risk assessment, where historical environmental behaviour is used to establish a baseline for individual locations. Current environmental conditions can then be compared against these historical patterns to identify abnormal changes. This allows the system to provide a more context-aware risk assessment rather than applying identical thresholds to every geographical area.

The early-warning component converts the estimated risk into progressive warning levels that are easier for users to understand. The system can provide supporting information about the factors contributing to the current risk level, improving interpretability and transparency. Historical-event evaluation is included to examine whether the generated warnings are sufficiently early, consistent and useful.

The project also includes a scenario-analysis module that allows hypothetical environmental changes to be explored. Users can modify selected environmental parameters and observe how the predicted risk changes under different conditions. A web-based visualization layer is included to present location information, risk levels, environmental conditions, warning states and prediction results in an accessible format.

The current scope does not claim to replace official disaster-management systems or professional geological assessment. Real-time deployment, large-scale sensor integration, automated emergency communication and operational decision-making are considered possible future extensions. The framework therefore provides a foundation that can be further developed with real-time data streams, weather forecasts, remote-sensing information and additional geographical features after appropriate validation.

1.7 APPLICATIONS

The proposed framework can support disaster-management research, environmental monitoring, landslide risk analysis, geographic visualization, early-warning research and scenario-based decision support[23]. It can also provide an educational and analytical platform for understanding how environmental changes influence model-based landslide risk.

The system can be used by researchers to study the relationship between environmental conditions and historical landslide occurrences. By combining temporal environmental information with machine-learning techniques, researchers can analyse how changes in rainfall and other factors are associated with variations in landslide risk[15]. The framework can also serve as a test environment for comparing different prediction algorithms and feature-engineering techniques.

For disaster-management applications, the system can provide a structured representation of changing risk conditions at selected locations. Progressive warning levels can help users understand whether the current conditions represent normal, moderately unusual or highly unusual environmental behaviour. The location-based interface can also make the prediction results easier to interpret geographically.

The framework can support environmental monitoring by continuously analysing available observations when real-time data sources are connected in future implementations[17]. Historical baselines can be used to identify deviations from normal environmental patterns, which can provide additional information for monitoring potentially hazardous conditions.

Another potential application is scenario-based decision support. Users can modify selected environmental parameters through what-if analysis and observe how the predicted risk changes[20]. This can help researchers understand model sensitivity and examine different environmental situations without waiting for an actual event to occur.

The system can also be useful for academic and educational purposes. Students and researchers can use the platform to understand data preprocessing, temporal feature engineering, machine-learning prediction, anomaly detection, risk classification and geographic visualization within a single application[18]. The modular architecture further allows individual components to be modified or replaced for experimental purposes.

In the future, the framework can be extended to integrate weather forecasts, remote-sensing data, IoT-based sensors, soil-moisture measurements and other geospatial information. Such extensions could enable more comprehensive environmental monitoring and support the development of real-time landslide early-warning applications after appropriate validation. Thus, the proposed system provides a flexible foundation for research, analysis, visualization and future development in landslide-risk monitoring.

The framework can also be applied in academic research and educational environments to study the relationship between environmental conditions and landslide occurrence. Researchers can use the system to compare different machine-learning models, analyse temporal rainfall patterns, study environmental anomalies and evaluate the effectiveness of different warning thresholds[16]. The visualization component can help students and researchers understand how raw environmental data is transformed into risk predictions and warning levels.

In future, the system can be extended by integrating real-time weather observations, rainfall forecasts, soil-moisture sensors, satellite data and other geospatial information. These additional data sources can improve the temporal and spatial coverage of the framework and support more comprehensive risk monitoring. With sufficient historical validation and reliable real-time data, the proposed framework can serve as a foundation for developing practical landslide early-warning and decision-support applications.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

Landslide early-warning research has developed through several major approaches, including rainfall thresholds, territorial warning systems, local monitoring, remote sensing, susceptibility mapping and machine-learning-based prediction. Each approach addresses a different aspect of landslide risk. Rainfall threshold methods attempt to identify environmental conditions under which landslides are more likely. Machine-learning approaches use historical observations and environmental features to learn relationships between input variables and landslide occurrence[2]. Monitoring-based systems use local observations to identify changes in slope or environmental conditions. The literature indicates that effective early-warning systems require not only prediction but also appropriate monitoring, warning decision rules and performance evaluation. These observations motivate the integrated architecture adopted in this project.

Recent research has also emphasized the importance of combining static susceptibility information with dynamic environmental conditions to improve the timeliness of landslide warnings. Historical terrain characteristics can indicate areas that are naturally more susceptible, while changing rainfall and soil conditions can provide information about short-term variations in risk. The integration of these complementary sources can therefore provide a more comprehensive representation of landslide risk[19]. In addition, interpretable prediction methods and user-oriented warning levels are increasingly important because risk information needs to be understandable for practical decision support. These research directions support the development of a location-adaptive framework that combines historical patterns, temporal environmental changes, machine-learning prediction and progressive warning generation.

2.2 TERRITORIAL EARLY-WARNING SYSTEMS

Territorial landslide early-warning systems generally operate over geographical regions and use environmental information such as rainfall to issue warnings[16]. Such systems demonstrate the importance of combining monitoring data with warning decision rules. However, regional systems may require carefully designed thresholds and appropriate spatial representation because environmental

behaviour varies between regions. This motivates the inclusion of location-adaptive analysis in the proposed framework.

Regional warning systems can provide useful information for areas where landslide activity is influenced by common environmental conditions[20]. Rainfall measurements collected across a region can be analysed to identify periods of increased landslide susceptibility[23]. However, regional-scale assessment may not fully represent local variations in terrain, soil properties, drainage conditions and rainfall patterns. Two locations within the same region may therefore experience different levels of risk under similar weather conditions. This creates a need for approaches that can adapt risk assessment according to the historical behaviour of individual locations. Location-specific baselines can help identify whether current environmental conditions are unusual compared with the normal conditions of that area. Such an approach can also reduce dependence on a single fixed threshold applied uniformly across a large geographical region. Combining regional monitoring with location-adaptive analysis can provide additional context for interpreting changes in risk. Geographic visualization can further help users understand where elevated conditions are occurring[20]. These considerations support the proposed framework's focus on location-aware risk estimation, temporal environmental analysis and progressive warning generation.

2.3 RAINFALL THRESHOLD METHODS

Rainfall thresholds are widely studied for rainfall-induced landslide warning. These methods generally analyse relationships between rainfall intensity, duration and landslide occurrence[17]. Although rainfall thresholds are useful, their performance can depend on the region, dataset, rainfall measurement source and characteristics of the landslide inventory. Therefore, a threshold developed for one region may not necessarily be directly applicable to another location. The proposed project does not replace rainfall analysis. Instead, rainfall is incorporated as an environmental factor within a broader AI and temporal analysis framework.

Rainfall-based analysis can be further improved by considering both short-term and accumulated rainfall conditions[5]. Recent rainfall may indicate immediate triggering conditions, while cumulative rainfall can represent the gradual increase in soil saturation. Antecedent rainfall is also relevant because rainfall occurring over previous days may influence the moisture condition of the slope. Incorporating these temporal characteristics allows the system to represent changing environmental conditions rather than relying on a single rainfall measurement.

Machine-learning models can use these features along with other available environmental variables to identify complex relationships that may not be captured by a simple threshold. The proposed framework can therefore treat rainfall thresholds as an important reference while allowing the prediction model to consider additional information[16]. Location-specific historical rainfall patterns can also be used to determine whether the current rainfall level is unusually high for a particular area. This approach provides greater flexibility when environmental behaviour differs across locations. The combination of rainfall analysis, temporal features, anomaly detection and machine-learning prediction forms an important part of the proposed landslide early-warning framework.

2.4 MACHINE-LEARNING-BASED LANDSLIDE ANALYSIS

Machine-learning methods have been applied to landslide susceptibility and prediction using environmental, terrain and geographical features. Models such as Random Forest, Support Vector Machine, Logistic Regression and Gradient Boosting can identify nonlinear relationships between input variables and target outcomes. Machine learning provides flexibility when multiple environmental variables are available[20]. However, the quality of a machine-learning result depends strongly on data quality, feature selection, temporal separation and leakage prevention. The proposed system therefore treats the AI model as one component of the complete warning architecture rather than considering it as the entire early-warning system.

Machine-learning-based landslide prediction can benefit from combining different types of environmental and geographical information. Features such as rainfall history, cumulative rainfall, soil moisture, elevation, slope and geographical location can provide complementary information for risk estimation[22]. Feature engineering is particularly important because raw environmental measurements may not directly represent the conditions associated with landslide occurrence. Temporal features can help the model capture changes occurring over hours, days or longer periods[20]. Model validation should also consider the temporal nature of the data to avoid using future information during training. Appropriate train-test separation is therefore important for obtaining a realistic estimate of model performance. In addition, class imbalance may occur when landslide events are less frequent than non-landslide conditions, requiring suitable evaluation measures. Probability-based predictions can then be converted into meaningful risk categories using predefined decision rules. This allows the AI model to contribute to a broader system that focuses not only on prediction accuracy but also on interpretability, temporal behaviour and practical warning usefulness.

2.5 MONITORING AND LOCATION-ADAPTIVE ANALYSIS

Local monitoring strategies demonstrate that landslide behaviour can be strongly dependent on the characteristics of individual sites. Environmental variables may have different meanings depending on the normal conditions of a location[16]. This concept motivates the historical baseline component of the proposed system. Instead of interpreting an environmental value only by its absolute magnitude, the system considers how the value compares with historical observations for the corresponding location.

A historical baseline can provide a reference representation of the normal environmental behaviour of a selected location. Such a baseline may be constructed using observations collected over an appropriate historical period[12]. Current measurements can then be compared with the corresponding historical values to identify deviations from normal conditions. This approach is useful because the same rainfall amount may represent different levels of environmental stress in different locations. Location-specific analysis can therefore provide additional context to the machine-learning prediction. The baseline can also be updated when sufficient new observations become available, allowing the system to adapt to changing environmental patterns. Temporal windows can be used to analyse recent conditions and compare them with similar historical periods. Anomaly scores can be generated to indicate the degree to which current conditions differ from the established baseline. These scores can be considered together with the model-generated probability when determining the overall warning state. Such integration can help reduce dependence on a single environmental variable or fixed threshold. The approach also provides a transparent way of explaining why a location may move from one warning state to another[11]. By combining local historical behaviour with current observations, the proposed framework aims to provide a more context-aware representation of landslide risk. This location-adaptive component forms an important connection between environmental monitoring, machine-learning prediction and progressive early-warning generation.

2.6 PERFORMANCE EVALUATION OF EARLY-WARNING SYSTEMS

Warning-system evaluation requires event-oriented measures to determine whether warnings provide useful information before hazardous events. Important measures include event detection, missed events, false warnings, and warning lead time, as these indicators assess both the effectiveness and operational usefulness of an

early-warning system[10]. Standards for territorial landslide early-warning systems emphasize systematic performance assessment using historical events and warning outputs. This event-based evaluation provides a more meaningful assessment of the early-warning framework by examining not only whether the AI model produces a high-risk probability, but also whether that probability is converted into an actionable warning before an actual landslide event. The evaluation stage therefore connects the AI prediction component with the practical objective of landslide early warning and helps identify areas where the warning thresholds, temporal risk escalation, and location-adaptive analysis may require further improvement. The proposed project therefore includes a dedicated evaluation stage in which generated warning states are compared with historical landslide events to measure detection performance, missed events, false warnings, and lead-time characteristics [6].

Event-based evaluation can also help distinguish between a technically accurate prediction model and a practically useful warning system. A model may produce accurate probability estimates while still generating warnings too close to the time of an event. Therefore, the timing and duration of each warning should be considered along with classification performance. The evaluation can examine whether a warning was issued sufficiently before the corresponding historical event and whether the warning remained active for an appropriate period. Repeated warnings for the same event can also be analysed to avoid counting multiple alerts as independent successful detections. False warnings should be examined because excessive alerts may reduce the usefulness and trust of an early-warning system. Similarly, missed events are important because failure to generate a warning before an observed landslide represents a significant limitation[19]. The evaluation can therefore provide a balanced view of both warning sensitivity and warning frequency. Different warning thresholds can be tested to understand their effect on detection, false-warning rates and lead time. Temporal risk escalation can also be evaluated by examining how quickly the system moves from lower warning states to higher warning states before an event. Location-adaptive thresholds can be compared with common thresholds to understand their effect on individual locations. These analyses can help identify suitable decision rules for converting predicted probabilities and environmental anomalies into warning states. Overall, event-oriented evaluation provides a direct connection between model outputs and the practical requirements of an early-warning system. It also provides a structured basis for improving the proposed framework through iterative testing and validation [6].

2.7 LITERATURE SURVEY

Research Area	Main Approach	Contribution
Territorial Early Warning	Regional rainfall-based warning	Framework for territorial warning systems
Rainfall Thresholds	Satellite-derived rainfall	Rainfall-based warning threshold analysis
Landslide Susceptibility	Machine learning	ML-based susceptibility modelling
Warning Performance	Event-oriented evaluation	Performance assessment of warning systems
Local Monitoring	Monitoring strategies	Location-specific landslide monitoring

Table 2.1 Literature Survey

2.8 RESEARCH GAP

The reviewed approaches demonstrate that rainfall analysis, machine learning, local monitoring and warning evaluation each provide important capabilities. However, these components are often treated as separate aspects of landslide research. The proposed framework combines them into a single pipeline. It uses AI to estimate risk, compares environmental conditions against location-specific historical behaviour, identifies abnormal environmental changes, monitors temporal risk escalation, generates progressive warning states and evaluates the resulting warnings against historical events[14]. The framework additionally provides what-if scenario analysis and an integrated dashboard, thereby connecting prediction, warning generation, evaluation and visualization.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 EXISTING SYSTEM

The Existing landslide assessment systems commonly use susceptibility maps, rainfall thresholds, historical event inventories, monitoring sensors or machine-learning models. Susceptibility maps are useful for identifying areas that have favourable conditions for landslide occurrence, but they generally represent relatively stable spatial characteristics rather than rapidly changing environmental conditions. Rainfall threshold systems provide temporal warning information but can depend strongly on regional characteristics and threshold selection[9]. Machine-learning systems can process multiple features and identify complex patterns, but a prediction probability alone does not necessarily provide a complete warning decision. Some monitoring systems provide highly valuable local information but may require dedicated sensors and infrastructure. Event-based evaluation may also be implemented separately from the prediction system. These limitations indicate the need for a framework that combines spatial context, temporal environmental behaviour, AI prediction, warning escalation and event-based evaluation[10-12].

Existing approaches therefore tend to focus on one or more specific aspects of landslide assessment rather than integrating the complete warning workflow into a single framework. A susceptibility map may identify potentially vulnerable regions but may not reflect sudden changes in rainfall or soil conditions[4]. Similarly, a fixed rainfall threshold may provide a simple warning rule but may not adequately represent differences between individual locations. Machine-learning models can improve prediction by combining multiple variables, but their outputs still need to be interpreted and converted into meaningful warning states. Local monitoring can provide detailed information, although the availability and maintenance of monitoring infrastructure may limit its application in some areas. Another challenge is the separation between prediction and evaluation, where model performance may be measured without considering whether warnings were issued early enough to be useful. These observations highlight the importance of integrating prediction, monitoring and evaluation within a common architecture. The proposed framework addresses this gap by combining historical environmental baselines with temporal feature analysis and machine-learning risk estimation. Environmental anomaly detection provides additional information about unusual changes at individual locations[20-23]. The risk score can then be transformed into progressive warning states based on defined

decision rules. The system also incorporates event-based evaluation to examine the relationship between generated warnings and historical landslide occurrences. This integrated approach is intended to provide a more complete research framework for studying location-adaptive landslide early warning while retaining the flexibility to incorporate additional environmental data in future implementations.

3.2 PROPOSED SYSTEM

The proposed system integrates eight sequential development phases into a single architecture. Historical landslide event data and environmental observations are first validated and processed. Temporal features are generated to represent previous environmental conditions. The AI risk model estimates a probability-based risk value. The location-adaptive module then constructs historical environmental baselines and compares current observations against the corresponding local behaviour. An abnormality score provides additional contextual information about unusual environmental conditions[5]. The warning engine combines predicted risk with abnormality, temporal trend and persistence to generate progressive warning states. The generated warning events are evaluated against historical landslide occurrences to determine detection, missed events, false warnings and lead-time characteristics. The what-if simulation module evaluates how controlled changes in rainfall and soil moisture affect model predictions. Finally, the outputs of the complete pipeline are made available through a React-based dashboard supported by FastAPI[15],[16].

The modular architecture allows each phase to be developed, tested and evaluated independently while maintaining a clear connection between the individual components. The data-validation stage ensures that unreliable or inconsistent observations are identified before they are used for model development. Preprocessing and feature-engineering stages transform the available information into a structured format suitable for temporal analysis and machine-learning prediction[17]. The risk model provides the primary probability-based estimate, while the location-adaptive module adds contextual information based on the historical behaviour of the selected area. The abnormality score helps identify whether current environmental conditions differ significantly from the established local baseline. Temporal trend and persistence information are used to avoid making warning decisions based only on a single observation. The warning engine therefore provides a progressive representation of changing risk rather than a single static classification. Each generated warning can be stored as an event and compared with historical landslide records during evaluation. The evaluation results can be used to identify weaknesses such as delayed detection, missed events or excessive false warnings[14]. The what-if simulation component

provides an additional method for analysing model sensitivity to changes in environmental conditions. Users can modify selected parameters and observe the corresponding change in predicted risk without altering the underlying historical dataset. The final dashboard integrates these outputs into a user-oriented interface where locations, risk values, warning states, environmental conditions and simulation results can be viewed together. This architecture provides a structured connection between data processing, AI prediction, location-specific analysis, warning generation, evaluation and visualization. The modular design also allows future components such as real-time weather APIs, satellite observations or sensor data to be incorporated without redesigning the entire system[13].

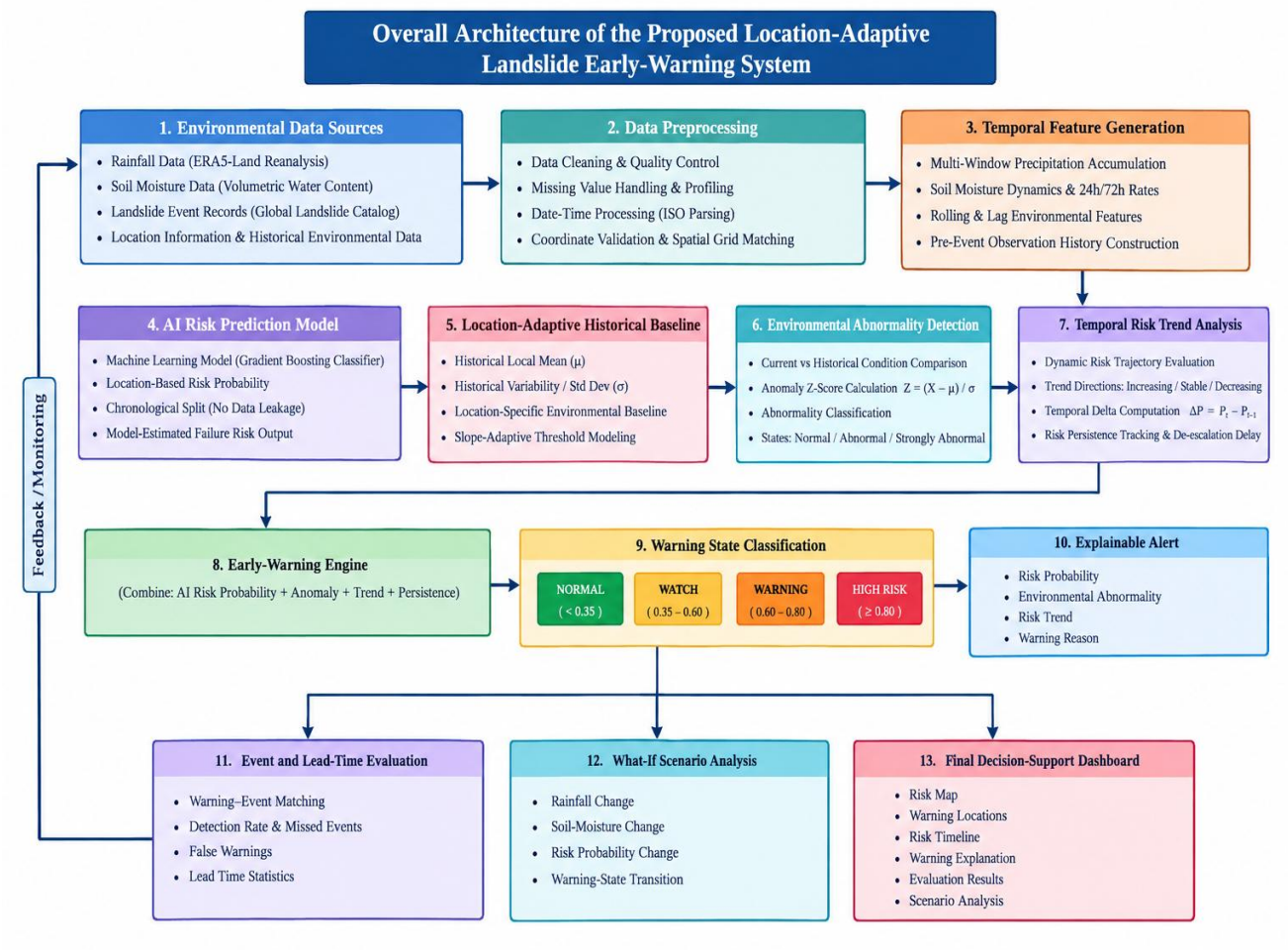


Figure 3.1. Overall Architecture of the Proposed Location-Adaptive Landslide Early-Warning System

CHAPTER 4

SYSTEM SPECIFICATION

4.1 HARDWARE REQUIREMENTS

The proposed system can be developed and executed using a standard modern computer. Since the project includes data processing, machine-learning experiments, visualization and web application development, sufficient memory and storage are required for handling environmental datasets and generated outputs[3].

A minimum of an Intel Core i5 or equivalent processor is recommended for performing data preprocessing, feature engineering and machine-learning operations efficiently. A minimum of 8 GB RAM is suitable for basic development, while 16 GB RAM is recommended for handling larger datasets and running multiple development tools simultaneously[4].

A standard monitor with a minimum resolution of 1366 × 768 is sufficient for developing and testing the web-based dashboard. A keyboard and mouse are required for general development and system interaction[2]. A dedicated GPU is not mandatory for the proposed system because the initial machine-learning models are designed to run on a standard CPU-based system. However, additional computational resources can be used in future if more complex deep-learning models or large-scale environmental datasets are incorporated[16].

Component	Requirement
Processor	Intel Core i5 or equivalent
RAM	Minimum 8 GB
Storage	Minimum 20 GB available
Operating System	Windows / Linux / macOS
Network	Internet connectivity
Display	HD or higher

Table 4.1 Hardware Requirements

4.2 SOFTWARE REQUIREMENTS

Python forms the primary computational environment because it provides libraries for data processing, numerical computation, machine learning, visualization and testing. Pandas and NumPy are used for data manipulation, while Scikit-learn supports machine-learning operations[9]. Matplotlib is used for generating analytical figures. The web application is implemented using React and TypeScript with Vite as the frontend build environment. FastAPI provides the backend API layer, while Leaflet supports geographic visualization. Pytest is used for automated software testing[12].

Software	Purpose
Python	Data processing and AI pipeline
Pandas	Data manipulation
NumPy	Numerical computation
Scikit-learn	Machine learning
Matplotlib	Visualization
PyYAML	Configuration
Pytest	Testing
React	Frontend
TypeScript	Frontend development
Vite	Frontend build
FastAPI	Backend API
Leaflet	Map visualization
Git	Version control

Table 4.2 Software Requirements

4.3 DATASET SPECIFICATION

The historical landslide event dataset used in the project contains 11,033 records and 31 variables.The dataset was subjected to systematic quality checks before being used for further processing. Duplicate records, date validity, coordinate information and missing values were examined. The project identified 0 duplicate rows and 10,895 unique latitude-longitude pairs. The event-date field contained multiple formats, which required mixed-format parsing during preprocessing[5].

After the corrected date-processing procedure, all 11,033 event dates were successfully parsed without fabricating or inferring missing dates. The historical landslide event dataset used in the project contains 11,033 records and 31 variables. The dataset was subjected to systematic quality checks before being used for further processing. Duplicate records, date validity, coordinate information and missing values were examined[3].

The project identified 0 duplicate rows and 10,895 unique latitude-longitude pairs. The event-date field contained multiple formats, which required mixed-format parsing during preprocessing. After the corrected date-processing procedure, all 11,033 event dates were successfully parsed without fabricating or inferring missing dates. The validated date information provides a reliable temporal basis for analysing the occurrence of landslide events[13]. The latitude and longitude fields were also examined to ensure that geographical information was available for spatial analysis. These coordinates allow the system to associate historical landslide events with specific locations and environmental conditions[4].

The cleaned dataset can therefore be used for temporal feature generation, location-based analysis and machine-learning model development. Additional preprocessing steps were applied to standardize data types and prepare the variables for subsequent analysis. The quality-checked dataset was then used as the foundation for integrating environmental variables such as rainfall and soil-related conditions. This preprocessing stage helps reduce errors and improves the reliability of the subsequent landslide risk prediction and early-warning analysis[21].

Attribute	Value
Records	11,033
Variables	31
Duplicate rows	0
Unique coordinate pairs	10,895
Valid event dates after processing	11,033

Table 4.3 Dataset Description

4.4 ENVIRONMENTAL DATA SPECIFICATION

Environmental data are incorporated to represent changing conditions associated with landslide risk. Rainfall and soil moisture are important temporal variables because changes in precipitation and antecedent wetness can influence slope conditions. The environmental processing stage aligns available environmental observations with the relevant geographical location and time[3]. Only information appropriate for the prediction time is considered, thereby reducing the possibility of temporal leakage. The environmental data are transformed into temporal features where supported by the available data. These features allow the AI model and subsequent warning modules to analyse changes rather than relying exclusively on isolated environmental measurements[13].

Environmental data are incorporated to represent changing conditions associated with landslide risk. Rainfall and soil moisture are important temporal variables because changes in precipitation and antecedent wetness can influence slope conditions. The environmental processing stage aligns available environmental observations with the relevant geographical location and time[5]. Only information appropriate for the prediction time is considered, thereby reducing the possibility of temporal leakage. The environmental data are transformed into temporal features where supported by the available data. These features allow the AI model and subsequent warning modules to analyse changes rather than relying exclusively on isolated environmental measurements. Short-term rainfall values can represent recent precipitation conditions, while cumulative and antecedent rainfall features can provide information about the wetness conditions that developed over a longer period[15].

The processed environmental variables are further used to understand the relationship between changing environmental conditions and historical landslide occurrences. Soil moisture information provides additional context by representing the degree of wetness in the surrounding area. Where sufficient historical observations are available, environmental values can also be compared with the normal behaviour of a particular location to identify unusual conditions[4],[7]. This location-based comparison supports the anomaly detection component of the proposed framework. The generated environmental features are passed to the machine-learning model and warning engine along with the relevant temporal and geographical information. This integrated processing helps the system produce more meaningful location-specific risk estimates and supports progressive warning generation based on changing environmental conditions[17].

4.5 DEVELOPMENT ENVIRONMENT

The development environment is organized as a modular project containing raw data, processed datasets, model files, configuration files, source code, tests, results and documentation. This organization allows each project phase to be developed and tested independently while maintaining clear interfaces between consecutive stages. The development environment is organized as a modular project containing raw data, processed datasets, model files, configuration files, source code, tests, results and documentation[18]. This organization allows each project phase to be developed and tested independently while maintaining clear interfaces between consecutive stages. The raw data directory stores the original datasets without modification, while processed data contains cleaned and transformed datasets used for analysis and model training. Model files store trained machine-learning models and related preprocessing components. Configuration files maintain project settings and parameters separately from the main source code[21].

The source-code structure is divided into modules for data preprocessing, feature engineering, prediction, anomaly detection, warning generation and evaluation. Test files are maintained to verify individual modules and reduce implementation errors. Results and visualization outputs are stored separately to support comparison and analysis during experimentation. Documentation contains project descriptions, methodology, configuration details and implementation notes. This modular structure improves maintainability, reproducibility and debugging, while also making it easier to extend the system with additional environmental variables, models and real-time data sources in future[25].

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 SYSTEM OVERVIEW

The complete implementation is divided into eight modules. The phases are connected sequentially, with the output of one phase becoming an input to the next stage. The implementation begins with project setup and dataset inspection, followed by preprocessing and temporal feature creation. The third phase develops the AI risk model. The fourth phase introduces location-specific historical baselines and abnormality analysis. The fifth module converts risk and temporal information into warning states. The sixth phase evaluates generated warnings against historical events. The seventh phase performs scenario-based what-if analysis, and the eighth phase integrates the outputs into a React and FastAPI web application.

During the first module, the project structure, datasets, dependencies and configuration settings are prepared and verified. The second phase focuses on cleaning the historical landslide and environmental datasets and generating suitable temporal features. Data validation is performed to identify missing values, duplicate records, invalid dates and inconsistent geographical information. In the third phase, selected machine-learning algorithms are trained using the processed features and their performance is compared using appropriate evaluation metrics. The trained model produces a probability-based estimate of landslide risk for the given conditions.

The fourth module adds location-specific analysis by comparing current environmental observations with historical patterns for the selected geographical area. Anomaly scores are generated to identify environmental conditions that differ significantly from the expected local behaviour. In the fifth phase, the predicted risk, anomaly information, temporal trends and persistence are combined to generate progressive warning levels such as Low, Moderate, High and Critical. The sixth phase evaluates these warnings using historical landslide events, considering detection, missed events, false warnings and warning lead time.

The seventh phase provides a what-if analysis module that allows controlled changes in rainfall and soil moisture conditions to be examined and their effect on predicted risk to be observed. Finally, the eighth phase integrates the complete processing pipeline into a web-based application. The React frontend provides an interactive interface for location search, risk visualization and warning information,

while the FastAPI backend handles data processing, model prediction and API communication. This phased architecture makes the system easier to test, maintain, debug and extend with additional environmental datasets or real-time data sources in future.

5.2 SYSTEM DESIGN

The overall data flow begins with historical landslide observations and environmental data. These inputs are cleaned and validated before temporal features are generated. The resulting features are passed to the AI risk model. The predicted risk is combined with location-adaptive environmental information and temporal trend analysis. Additionally, the system maintains historical warning records to identify warning transitions and evaluate lead time before landslide events. The simulation module applies hypothetical changes in rainfall and soil moisture to analyze possible changes in risk and warning states. All processed results are mapped integrated into the dashboard to support location-based visualization, historical assessment, evaluation, and decision support.The warning engine then generates a warning state and explanation. The resulting warning records are passed to the evaluation module, while baseline observations are used by the simulation module for scenario analysis. Finally, the outputs are exposed through backend APIs and displayed by the frontend dashboard.

The data flow is designed to maintain a clear separation between data processing, prediction, warning generation and visualization. Historical observations provide the reference information required for understanding previous landslide conditions. Environmental observations are synchronized with geographical and temporal information before being used for prediction. The temporal feature generation stage captures recent and accumulated environmental behaviour that may not be represented by a single observation. The AI model then transforms these processed features into a probability-based risk estimate. The location-adaptive module provides additional context by comparing current conditions with the historical behaviour of the selected area. The warning engine combines these outputs to determine the appropriate warning level and generates supporting information for the user.

The evaluation module stores and analyses warning outputs in relation to historical landslide events. This allows the system to examine whether warnings were generated before observed events and identify missed or unnecessary warnings. The simulation module provides an additional analytical layer by allowing users to examine how changes in environmental conditions could influence the predicted risk. The backend APIs act as the communication layer between the processing modules and the

frontend application. The React dashboard presents the final results through risk indicators, warning states, environmental information and location-based visualizations. This complete data flow ensures that the system operates as an integrated pipeline rather than as independent modules and provides a structured foundation for future integration of real-time environmental observations.



Figure 5.1. Project Data Processing Workflow

5.3 MODULE-1 PROJECT SETUP

Module 1 establishes the foundation of the project by creating the required directory structure, configuring the development environment and inspecting the primary dataset.Source-code modules, configuration files and automated tests are organized independently. The initial inspection confirmed the dataset dimensions and available variables. Data-quality checks were also performed to identify duplicates, missing information, date-format variations and geographical fields. The Module 1 implementation was validated using automated tests, with 9 out of 9 tests passing.

The module also established the basic configuration required for executing the remaining project phases. File paths, dataset locations and common project parameters were organized so that they could be accessed consistently by different modules. Automated testing was introduced at this stage to verify that the initial setup and dataset inspection functions operate as expected. The successful completion of all 9 tests provided an initial validation of the module implementation. Any invalid or inconsistent data identified during inspection was documented for handling during the preprocessing stage. The output of Module 1 therefore serves as the verified input for Module 2, where detailed data cleaning and temporal feature engineering are performed. This structured approach helps maintain reproducibility and provides a clear development trail for the complete landslide early-warning system.

5.4 MODULE-2 DATA PIPELINE

Module 2 converts raw event information into a validated dataset suitable for further analysis. The processing pipeline includes data-quality checking, date normalization, temporal feature generation and exploratory analysis. The original event dataset contains multiple date representations. A mixed-format parsing strategy was therefore used to avoid incorrectly discarding valid records. This resulted in all 11,033 event dates being successfully processed. Temporal features are created to represent previous environmental conditions and changes over time. Such features can include lagged values, rolling values and temporal differences depending on environmental-data availability. The Module also generates exploratory figures showing temporal and geographic characteristics of the processed dataset.

During preprocessing, numerical and categorical variables are examined to identify inconsistent formats and unsuitable values. Geographical fields are checked to ensure that latitude and longitude information can be used for spatial analysis. Date normalization provides a common temporal representation for combining landslide events with environmental observations. The generated temporal features help the model capture changes in environmental conditions rather than relying only on individual observations. Where rainfall data are available, cumulative and antecedent rainfall features can be generated to represent recent and previous precipitation conditions. Similarly, soil moisture observations can be processed to represent changes in ground wetness over different time periods.

Exploratory analysis is performed to understand the distribution of events across time and geographical regions. These visualizations help identify common patterns, unusual observations and possible data-quality issues before model training. The processed dataset is stored separately from the original raw data to preserve data

traceability. The module also ensures that preprocessing operations are applied consistently so that the same transformations can be used during later prediction stages. The output of Module 2 therefore provides a structured and validated dataset for the AI risk model, location-adaptive analysis and subsequent warning-generation modules. This stage forms an important connection between raw historical observations and the analytical components of the proposed landslide early-warning framework.

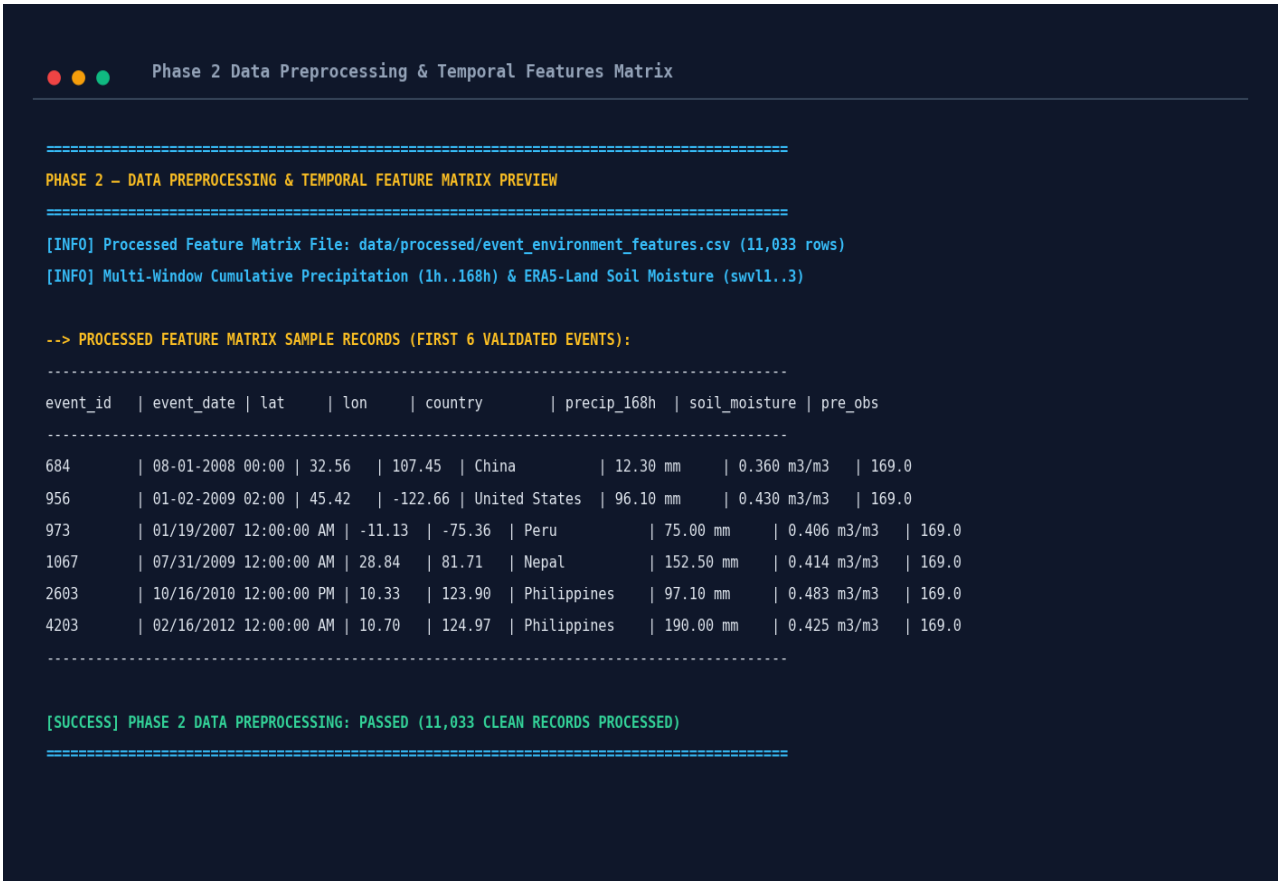


Figure 5.2. Data Preprocessing and Temporal Feature Generation Output

5.5 MODULE-3 AI RISK MODEL

Module 3 develops the machine-learning component responsible for estimating landslide risk. The processed temporal and environmental features are prepared for model training and evaluation. A major design consideration is preventing information leakage. Variables representing information that becomes available only after a landslide event are not used as predictive inputs. This ensures that the model does not gain access to information that would not be available during an actual prediction. Chronological separation is used to represent the temporal nature of the prediction problem. Earlier observations are used for training, while later observations are reserved for validation and testing. The trained model produces a probability-based

risk estimate that is subsequently passed to the warning engine.

The selected input features may include rainfall-related variables, soil moisture conditions, temporal features and relevant geographical information depending on data availability. Feature preparation is performed consistently across training, validation and testing stages to maintain reliable model behaviour. The target variable is defined according to the availability and representation of historical landslide events. Model training is performed using the prepared training data, while validation data are used to examine model performance during development. The final test period is kept separate to provide an independent assessment of predictive behaviour.

Model performance is examined using suitable classification and probability-based evaluation measures. The predicted probability provides more information than a simple binary output because it allows the warning engine to apply different risk levels. The probability value can subsequently be mapped into categories such as Low, Moderate, High and Critical according to the defined warning rules. Model outputs are also combined with anomaly information and temporal trends rather than being treated as the only source of warning information. This design allows the system to distinguish between model-based risk and unusual environmental behaviour.

The trained model and required preprocessing components are stored for later use by the prediction pipeline. During inference, new environmental observations are transformed using the same feature-processing approach used during training. The resulting features are supplied to the trained model to generate a risk probability for the selected location and time. This probability is then transferred to the location-adaptive and warning-generation modules. Thus, Module 3 provides the core predictive component while maintaining temporal consistency, leakage prevention and reproducible model processing.

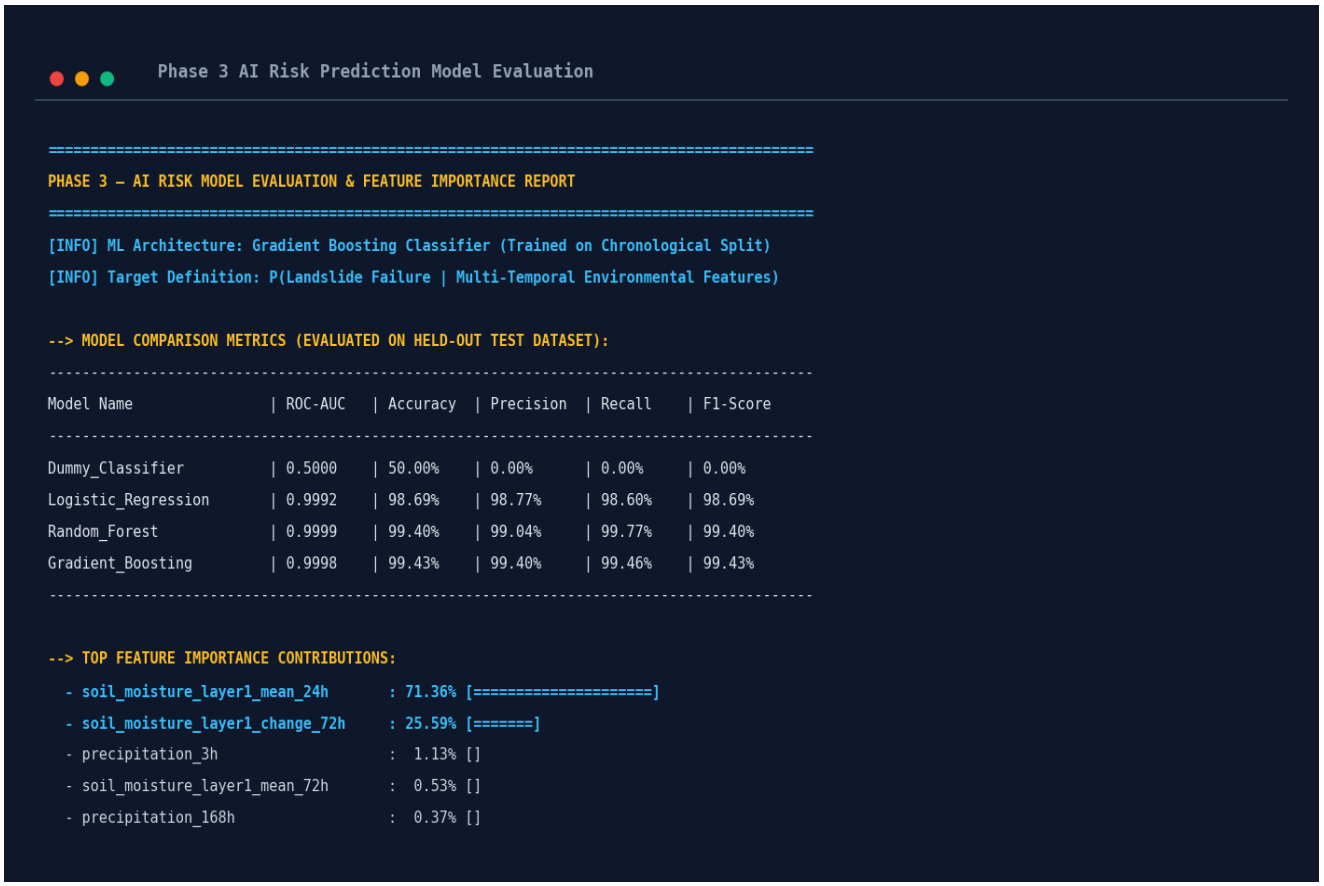


Figure 5.3. AI Risk Prediction Output

5.6 MODULE- 4 LOCATION-ADAPTIVE ANALYSIS

Module 4 introduces the location-specific environmental baseline. Historical environmental observations for a location are used to represent the normal behaviour of that location. The current observation is compared against the historical baseline to determine whether the environmental condition is within the expected range or represents an abnormal change. This approach is important because environmental values have different meanings in different geographical and seasonal contexts. A rainfall value that is common at one location may be unusual at another location. The abnormality information is used as supporting evidence for the warning system and is not treated as a direct guarantee that a landslide will occur.

The baseline can be developed using historical rainfall and soil moisture observations available for the selected geographical area. Statistical measures such as the historical mean, median, standard deviation or suitable percentile ranges can be used to represent normal environmental behaviour. The current observation is then evaluated against these reference values to calculate an abnormality measure. Seasonal patterns can also be considered where sufficient historical observations are available. This helps avoid treating naturally high rainfall periods as abnormal without

considering the historical context. The location-specific analysis therefore provides additional information that complements the machine-learning risk probability.

The generated abnormality score is passed to the warning engine along with the model prediction and temporal trend information. A persistent abnormal condition can provide stronger contextual evidence than a single isolated observation. The module also supports geographical comparison by allowing environmental behaviour to be examined across different locations. Historical baseline information is stored separately so that it can be reused during future prediction requests. This module therefore improves the contextual interpretation of environmental conditions and supports more location-adaptive warning generation within the proposed framework.



Figure 5.4. Location-Adaptive Baseline and Environmental Abnormality Analysis

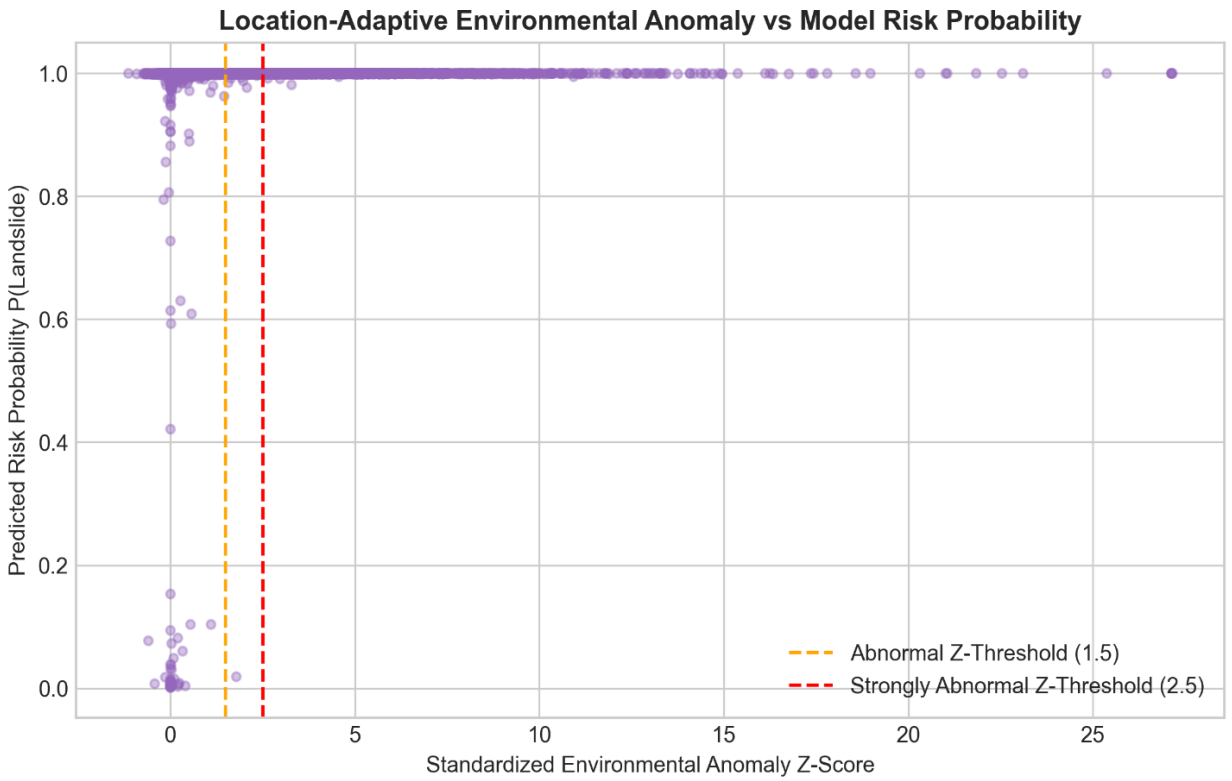


Figure 5.4. Location-Adaptive Baseline and Environmental Abnormality Analysis

5.7 MODULE-5 EARLY-WARNING ENGINE

Module 5 converts AI risk probability and environmental information into progressive warning states. The warning engine considers the current risk level, environmental abnormality, temporal trend and persistence. The configured warning states are **NORMAL, WATCH, WARNING and HIGH RISK**. The NORMAL state represents conditions where the available indicators do not show significant elevation in risk. The WATCH state indicates that environmental conditions or model predictions have started to move away from the normal range. The WARNING state represents a higher level of concern when multiple indicators provide supporting evidence of increasing risk. The HIGH RISK state is generated when the combined indicators reach the defined high-risk conditions. These states provide a structured representation of changing risk rather than displaying only a numerical probability.

The warning engine also considers the persistence of risk conditions to reduce unnecessary changes caused by short-term fluctuations. Temporal trends are used to identify whether risk is increasing, decreasing or remaining relatively stable. Environmental abnormality provides additional contextual information about unusual rainfall or soil moisture conditions at the selected location. The final warning state is generated according to predefined decision rules that combine these inputs. Each warning transition is recorded with the corresponding time, location, risk probability

and supporting information. This allows the system to track how warning conditions develop over time. The generated explanation provides users with a clearer understanding of the factors contributing to the current warning state. The warning output is then passed to the evaluation module and displayed through the web dashboard. This module therefore connects the machine-learning prediction with an interpretable and progressive early-warning mechanism.

State	Risk Probability
NORMAL	Below 0.35
WATCH	0.35 to below 0.60
WARNING	0.60 to below 0.80
HIGH RISK	0.80 and above

Table 5.2 Warning State Configuration

The system also tracks whether risk is increasing, stable or decreasing. Persistence logic is used to avoid treating every isolated elevated observation as a separate warning condition. The resulting warning output includes a warning state and an explanation describing the major contributing factors. This helps maintain consistent warning decisions across consecutive observations. It also supports more reliable interpretation of changing environmental conditions over time. The completed Module 5 pipeline generated 10,455 warning observations, 6,666 discrete warning events and 6,688 historical location summaries.



Figure 5.5. Early-Warning Engine Output

5.8 MODULE-6 EVENT AND LEAD TIME EVALUATION

Module 6 evaluates the usefulness of generated warnings by comparing warning events with historical landslide events. The event-matching process considers spatial proximity and temporal ordering. A warning must occur before the corresponding event to be considered a valid advance warning under the configured evaluation protocol. The evaluation separates detected events, missed events and false warning events. Lead time is calculated from the warning start time to the corresponding event time. This phase is important because a system intended for early warning must be evaluated in terms of whether warnings occur before actual events rather than only in terms of conventional classification metrics. The evaluation also measures the reliability of the warning sequence across different locations and event conditions. These results help determine whether the proposed system can provide meaningful advance warnings for historical landslide events.

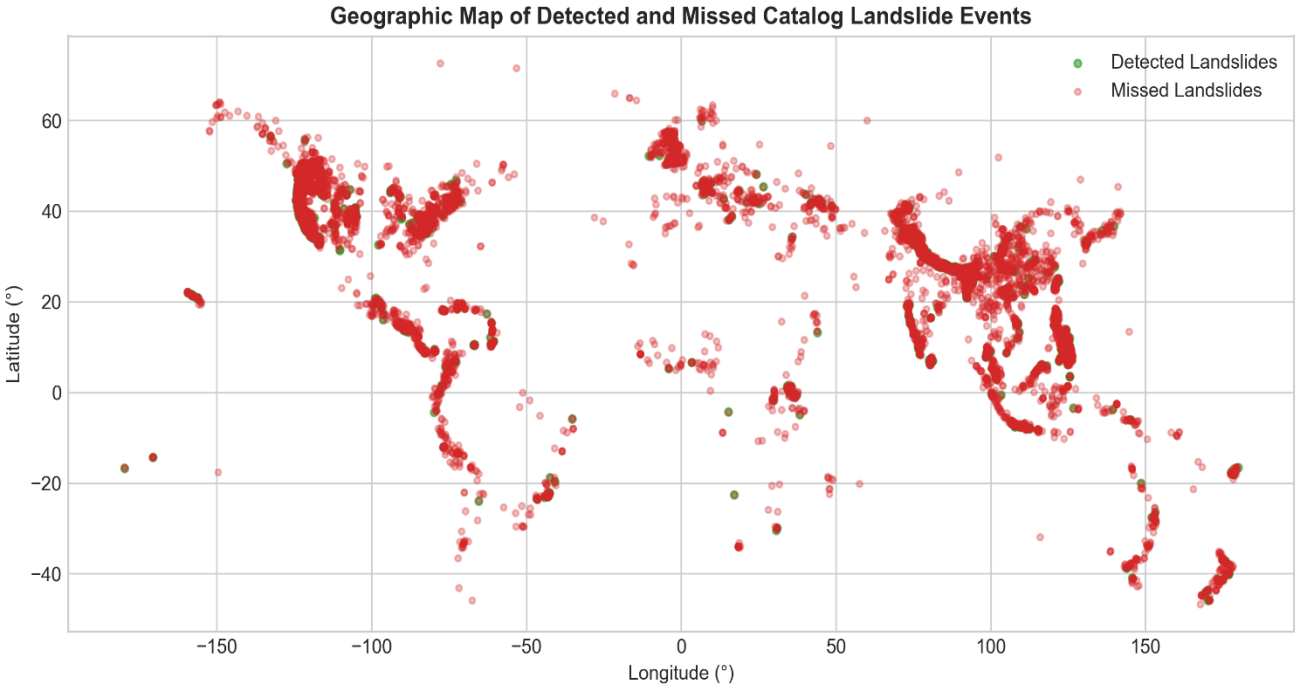


Figure 5.6. Event Matching and Lead-Time Evaluation Output

Important: Actual detection rate, precision, miss rate and lead-time values should be inserted from the Module 6 result files only.

5.9 MODULE-7 WHAT-IF SIMULATION

Module 7 provides scenario-based sensitivity analysis by modifying rainfall and soil-moisture conditions and observing the resulting model predictions. Rainfall scenarios include changes such as −25%, −10%, +10%, +25%, +50% and +100%. Soil-moisture scenarios include −10%, +10%, +25% and +50%. Combined scenarios can also be evaluated. The modified features are passed through the same prediction and warning pipeline used for baseline observations. The difference between baseline and scenario probability is then calculated. The simulation also checks whether the modified input remains within the model's represented training domain. This is important because extreme modifications may produce extrapolation cases. The simulation results are compared with the baseline to identify changes in risk probability and warning-state transitions. These results help assess how sensitive the warning system is to environmental changes and identify conditions that may push the model beyond its reliable training range.

The baseline observation acts as the reference point for every simulation scenario. For each scenario, the selected environmental variable is systematically increased or decreased while other relevant conditions are kept unchanged unless a combined scenario is specifically requested. This allows the effect of individual environmental changes to be examined more clearly. The resulting probability is compared with the original model output to determine the direction and magnitude of the change. A positive probability difference indicates an increase in predicted risk, while a negative difference indicates a reduction compared with the baseline condition.

The simulation module also records the corresponding warning state generated for each modified condition. This makes it possible to identify situations where a relatively small environmental change causes the system to move from NORMAL to WATCH, WATCH to WARNING, or WARNING to HIGH RISK. Such transitions provide useful information about the sensitivity of the warning rules. The module does not interpret a simulated scenario as an actual future event; instead, it provides a controlled analytical experiment based on the trained model. This distinction is important because the simulation depends on the assumptions and limitations of the available historical training data.

The training-domain check provides an additional reliability safeguard. If a modified rainfall or soil-moisture value falls substantially outside the range represented in the training dataset, the scenario is marked as an extrapolation case. These cases can be reported separately so that users do not interpret the resulting probability with the same confidence as an in-domain prediction. Scenario outputs can also be visualized using tables, probability comparisons and warning-state transitions. The results can help researchers understand the behaviour of the trained model under different environmental conditions and identify variables that have a stronger influence on predicted risk.

The what-if module can further support comparative analysis between locations by applying similar environmental changes to different historical baselines. This allows the system to demonstrate how the same percentage change may produce different predicted responses depending on the location-specific conditions. The generated simulation records are retained for later analysis and can be displayed through the web dashboard. Overall, Module 7 provides an interpretable sensitivity-analysis layer that complements the primary prediction and warning pipeline while explicitly considering the limitations of model extrapolation.



Figure 5.7. What-If Scenario Analysis Output

5.10 MODULE-8 WEB DASHBOARD

Module 8 integrates the complete project into a web-based dashboard. The frontend is implemented using React, TypeScript and Vite, while FastAPI provides the backend API layer. The dashboard contains separate views for overall risk information, geographic visualization, warnings, event evaluation, simulation and location-specific details. The map interface presents location-based information, while the warning page provides filtering and pagination. The evaluation page displays outputs from Module 6, and the simulation page provides scenario analysis based on Module 7. The application also includes loading, error and empty-data states to provide appropriate feedback to users. The dashboard connects the outputs of all previous modules into a single interface for easier analysis and interpretation. It also helps users access risk, warning, evaluation and simulation results in an organized manner.

The React frontend communicates with the FastAPI backend through defined API endpoints. The backend receives user requests, processes the required inputs and returns structured prediction or analysis results to the frontend. This separation between the frontend and backend improves modularity and allows the individual components to be developed and tested independently. The application can accept

location-related inputs and retrieve the corresponding risk information from the processing pipeline. The returned results are then converted into visual components such as risk indicators, warning labels, charts, tables and map-based information.

The risk view provides a summary of the current prediction along with relevant environmental information. The geographic view helps users understand the spatial distribution of locations and associated risk information. The warning view displays generated warning records and allows users to examine warning history using filtering and pagination options. This makes it easier to review multiple warning events without displaying all records simultaneously. The evaluation view presents event-based performance information, including warning outcomes and relevant evaluation measures generated by Module 6.

The simulation view allows users to select environmental scenarios and observe how changes in rainfall and soil moisture influence the predicted risk. Baseline and modified scenario results can be compared to understand probability changes and warning-state transitions. Location-specific information is presented separately so that users can examine the historical environmental behaviour associated with a selected area. The interface also provides suitable messages when data are unavailable, an API request fails or processing is still in progress.

The dashboard is designed to provide a consistent user experience across all project modules. Common components and API communication methods are reused where appropriate to reduce unnecessary duplication in the frontend implementation. The backend also acts as a controlled access layer between the user interface and the machine-learning and analytical modules. This prevents the frontend from directly handling internal model-processing operations. Overall, Module 8 converts the individual analytical components into an integrated research prototype that allows users to explore landslide risk, warning behaviour, historical evaluation and environmental scenarios through a single web-based interface.

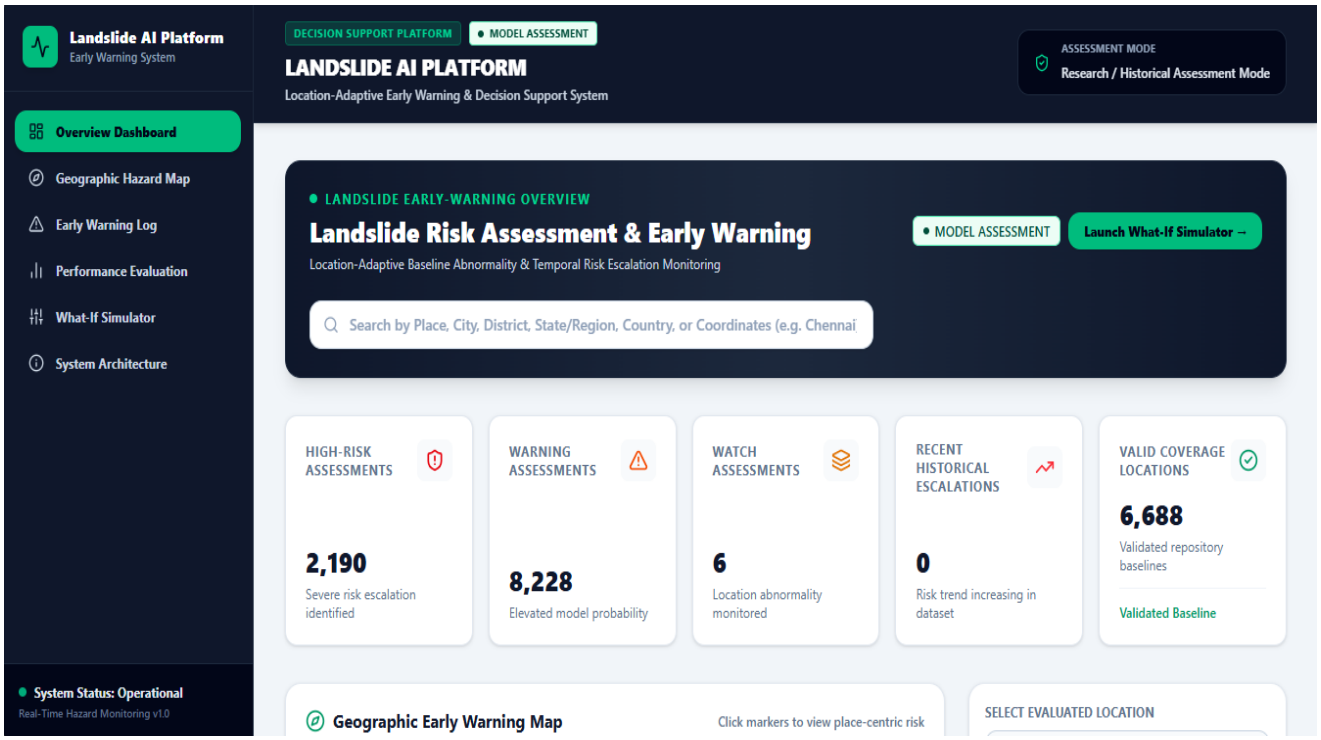


Figure 5.8. Final React-Based Landslide Early-Warning Dashboard

5.11 TECHNOLOGY STACK

The proposed system integrates Python-based data processing and artificial intelligence techniques with a modern web application architecture. Python is used to implement the data preprocessing, temporal feature generation, AI-based risk prediction, location-adaptive analysis, warning generation, evaluation, and simulation processes. React and TypeScript are used to develop the interactive frontend interface, while FastAPI provides the communication layer between the processed project outputs and the frontend through structured API endpoints. Leaflet is used for geographic visualization of location-based landslide risk and warning information. Pytest is employed for automated testing and validation of the implemented modules. This layered architecture separates computational processing from presentation, improves modularity and maintainability, and enables the dashboard to consume structured results through well-defined API endpoints.

The backend architecture acts as the central processing layer of the application. It receives requests from the frontend, validates the required parameters and invokes the appropriate analytical module based on the requested operation. The machine-learning model and supporting preprocessing components are maintained within the backend environment so that prediction operations remain consistent with the trained model configuration. The API responses are returned in structured formats that can be directly interpreted by the React frontend.

The frontend is responsible for presenting complex analytical outputs in a user-friendly manner. React components are organized according to individual dashboard functions, allowing risk information, warning records, evaluation results and simulation outputs to be displayed independently. TypeScript provides type checking and helps reduce errors during frontend development. Leaflet supports interactive map-based visualization, allowing geographical information to be displayed alongside the corresponding risk and warning states. This provides an additional spatial perspective for interpreting the model outputs.

The modular architecture also supports independent testing and future development. Individual Python modules can be tested using Pytest without requiring the complete web application to be executed. Similarly, frontend components can be modified without changing the underlying machine-learning implementation as long as the defined API structure is maintained. This separation reduces coupling between different parts of the system and makes debugging more manageable.

The architecture is also designed with future extensibility in mind. Additional environmental variables, machine-learning models, real-time data sources or external weather APIs can be incorporated without completely redesigning the application. The same API-based communication structure can be used to expose new analytical functions to the frontend. Overall, the selected technologies provide a modular foundation for integrating data processing, artificial intelligence, geographic visualization and web-based decision-support functionality within the proposed landslide early-warning framework.

The system architecture also supports reliable data flow between the analytical modules and the user interface. Each module produces structured outputs that can be reused by subsequent stages without unnecessary duplication of processing. The use of separate backend services and frontend components makes the application easier to update, test and maintain. The architecture can also support deployment in a cloud or local environment depending on project requirements. In future, real-time rainfall observations, soil-moisture measurements, weather forecasts and additional geographic datasets can be connected through the existing API layer. This provides a flexible foundation for extending the current research prototype into a more comprehensive environmental monitoring and landslide risk analysis platform.

CHAPTER - 6

RESULT AND DISCUSSION

6.1 DATASET PROCESSING RESULTS

The preprocessing stage successfully validated the primary landslide event dataset. The validation confirmed the consistency of important fields such as event dates and geographical coordinates, while duplicate records and data-quality issues were systematically examined. All available event dates were successfully converted into a consistent format without fabricating missing information. The cleaned dataset therefore provides a reliable foundation for temporal feature generation, environmental data integration and subsequent machine-learning analysis.

Parameter	Result
Total records	11,033
Total variables	31
Duplicate records	0
Unique coordinate pairs	10,895
Valid event dates	11,033

Table 6.1 Dataset Processing Results

The results demonstrate that the preprocessing pipeline preserved the available event records while resolving the multiple date-format issue. The dataset was found to contain no duplicate records, ensuring that each event was retained only once during processing.All 11,033 event dates were successfully validated and standardized into a consistent format.The coordinate validation confirmed 10,895 unique coordinate pairs for spatial analysis.These preprocessing steps improved the consistency and reliability of the dataset for subsequent analysis.The validated dataset was then used as the input for temporal feature generation and AI-based risk prediction.

The successful completion of preprocessing also ensured that the historical event information could be combined with environmental observations using a consistent temporal and geographical reference. The standardized dates provide a reliable basis for generating temporal features and analysing event patterns over time. The validated coordinate information supports location-specific analysis and the construction of historical environmental baselines. Since no duplicate records were

identified, the possibility of repeated events affecting subsequent analysis was reduced. Overall, the preprocessing results establish a structured and quality-checked dataset that can be reliably used by the remaining modules of the proposed landslide early-warning framework.

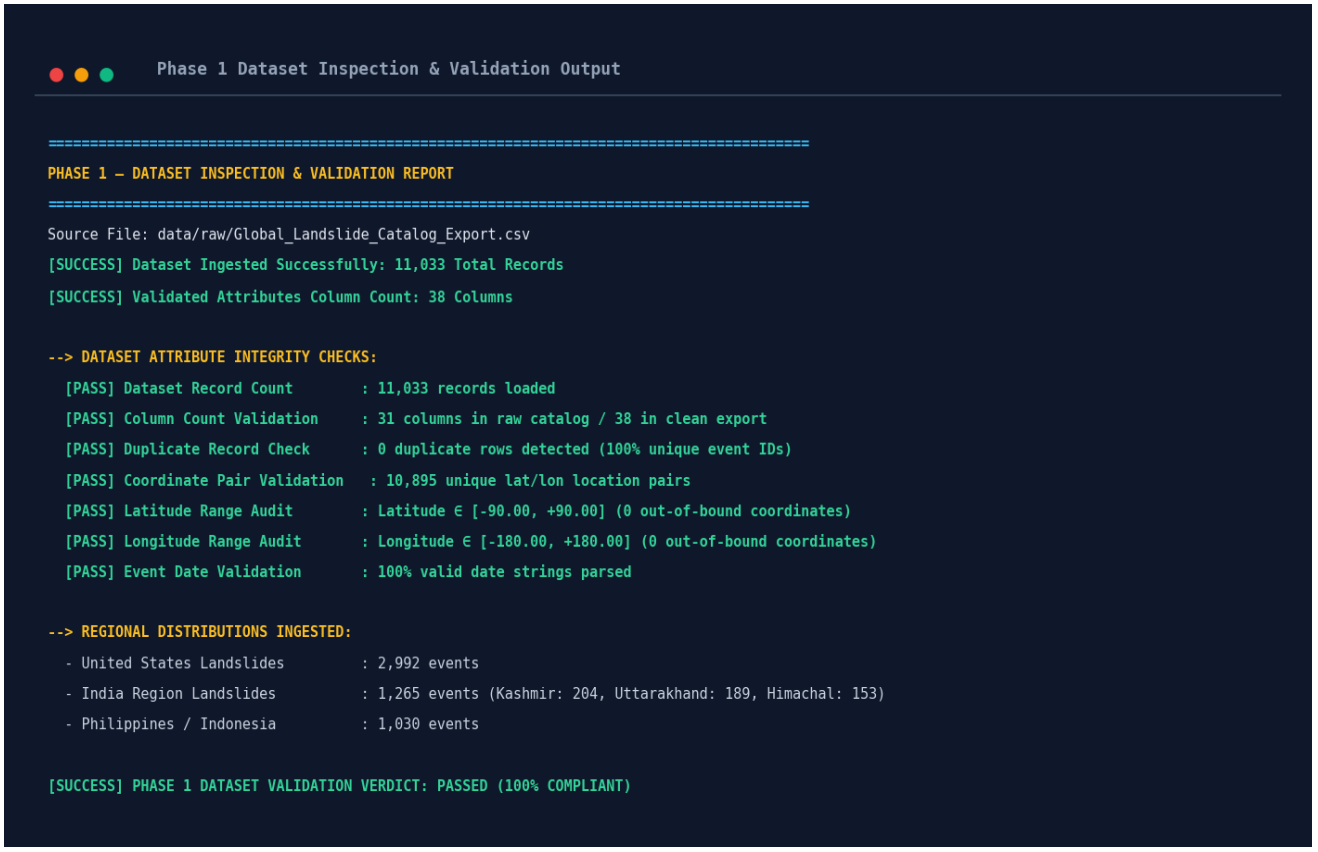


Figure 6.1. Dataset Validation Output

6.2 TEMPORAL FEATURE RESULTS

Temporal processing provides information about previous and changing environmental conditions. The generated features allow the system to consider environmental behaviour over time rather than relying only on a single observation. The temporal-processing output forms an important connection between raw environmental data and the AI risk model. It captures changes in rainfall, soil moisture, and other environmental variables across different time periods. These features help identify increasing or decreasing environmental trends that may influence landslide risk. The processed temporal features are then supplied to the AI model for risk prediction. This improves the system's ability to analyse changing conditions and support more meaningful risk assessment.

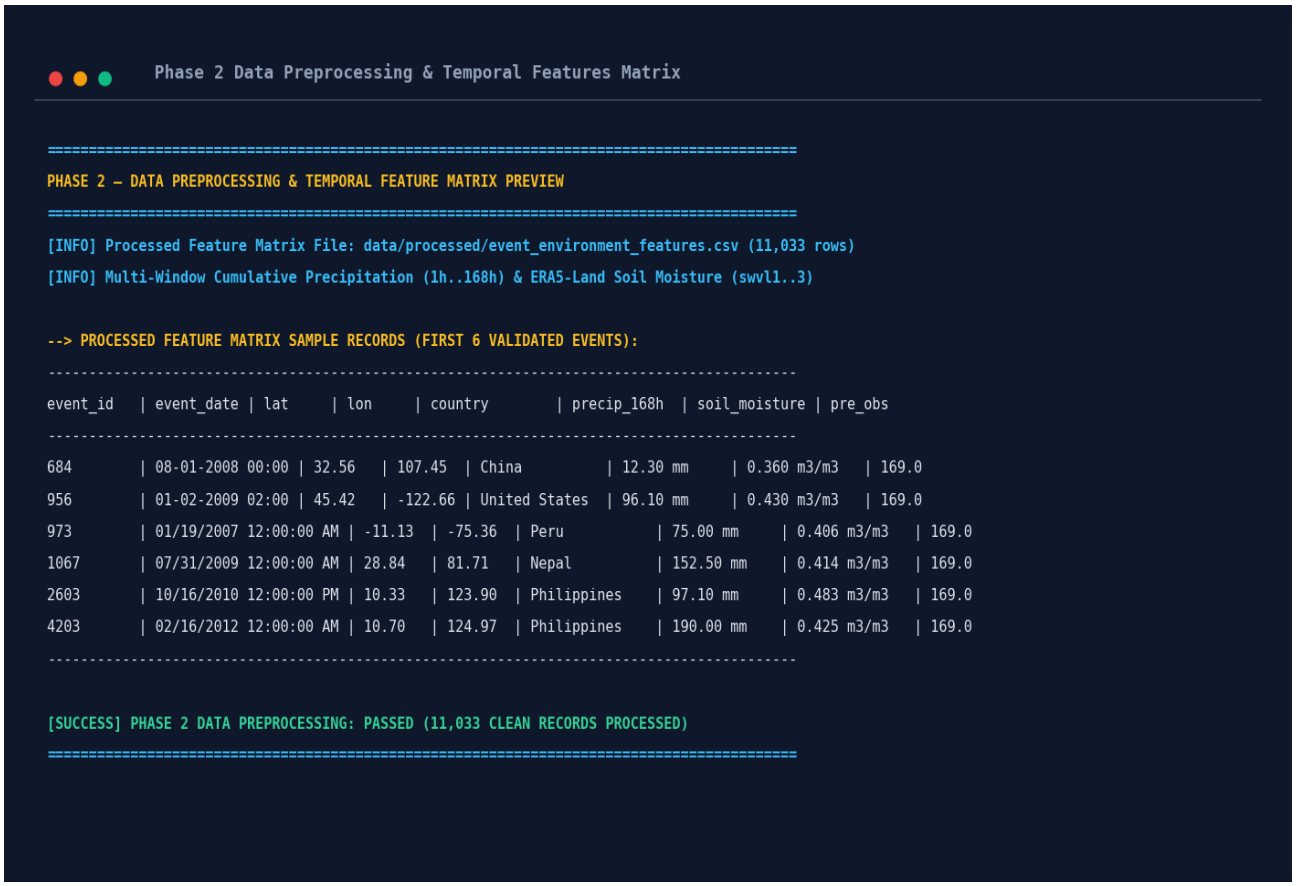


Figure 6.2. Temporal Feature Output

6.3 RISK PREDICTION RESULTS

The AI risk model produces probability-based risk estimates for processed observations. These values are subsequently used by the warning engine. The final report should include the actual model evaluation results obtained from the Phase 3 experiments. Accuracy, precision, recall, F1-score and AUC should only be reported when they are available in the actual project output. The model performance is assessed using the available evaluation metrics from the Phase 3 experiments. These results indicate how effectively the model distinguishes between different risk conditions.The predicted probabilities provide a continuous measure of landslide risk for each processed observation.These probability outputs are further integrated with the location-adaptive analysis and warning engine to generate the final warning state.

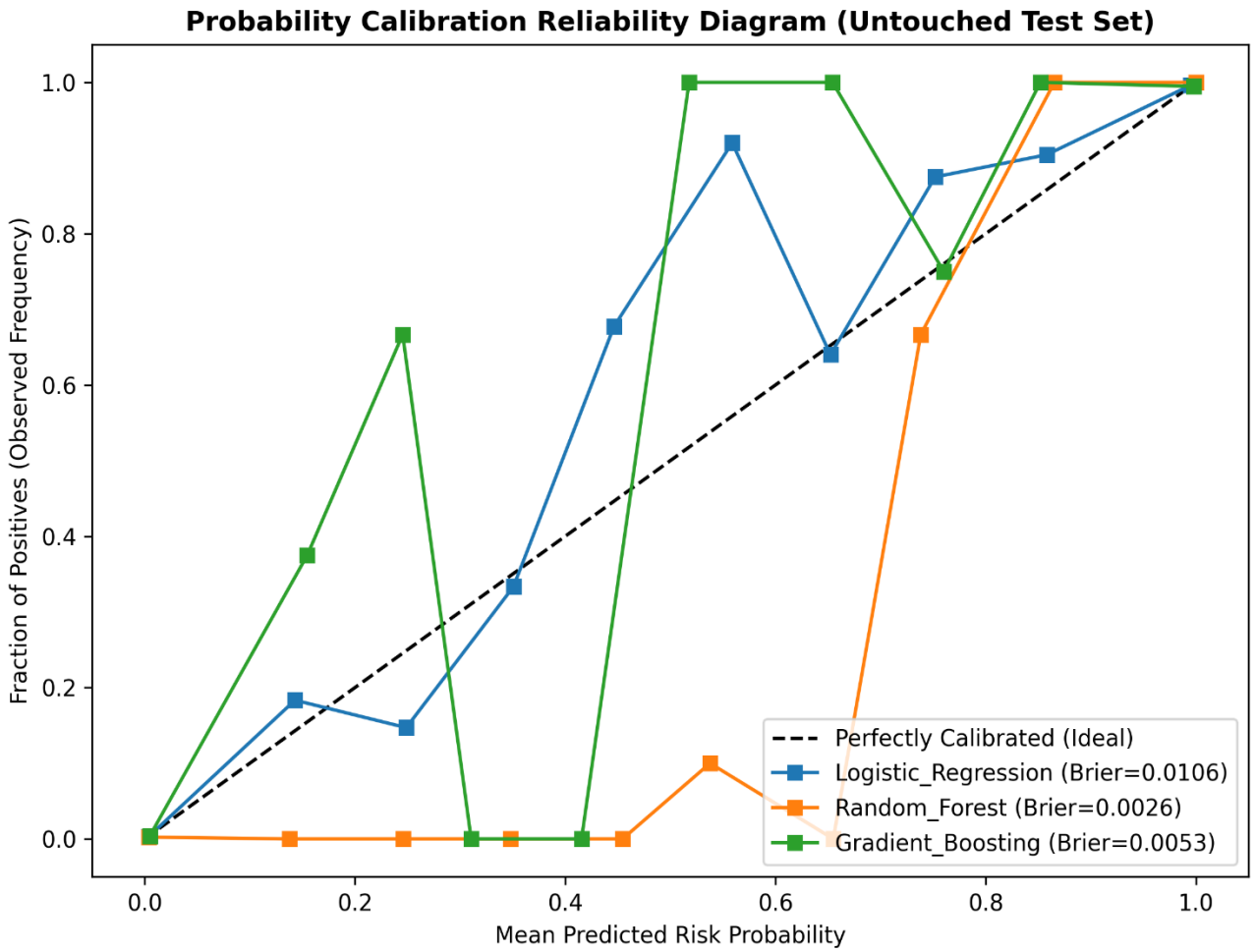


Figure 6.3. AI Risk Prediction Result

6.4 LOCATION-ADAPTIVE RESULT

The location-adaptive analysis compares current environmental conditions with historical observations for the corresponding location. This analysis provides additional context to the AI prediction. An environmental condition may be considered more significant when it represents an unusual deviation from the normal historical behaviour of that location.

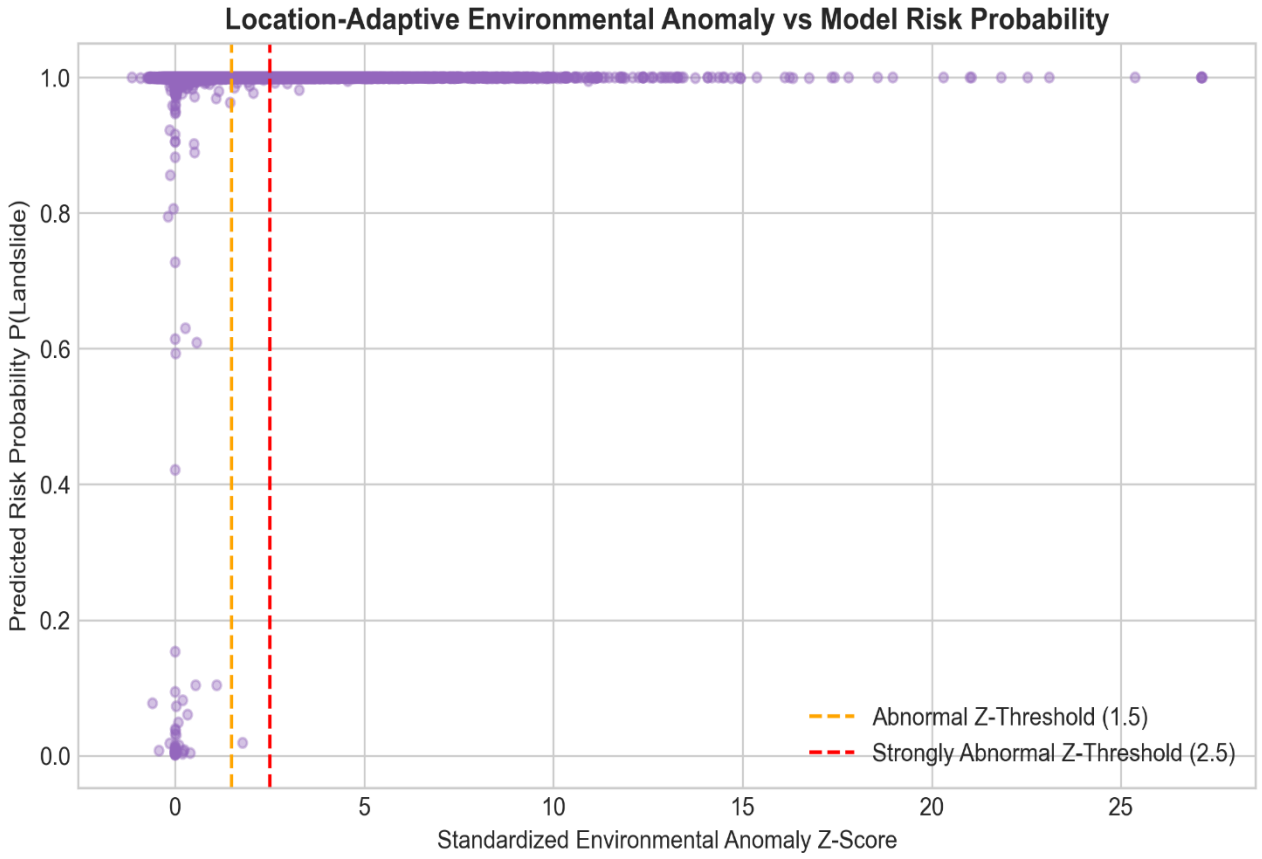


Figure 6.4. Location-Adaptive Environmental Analysis

6.5 EARLY-WARNING RESULT

The warning engine produced **10,455 observation-level warning records** and **6,666 discrete warning events**.

Warning State	Observations	Percentage
NORMAL	382	3.65%
WATCH	3,991	38.17%
WARNING	2,459	23.52%
HIGH RISK	3,623	34.65%
Total	10,455	100%

Table 6.2 Warning State Distribution

The distribution demonstrates the operation of the progressive warning-state mechanism. The values represent processed project observations and should not be interpreted as current live forecasts.

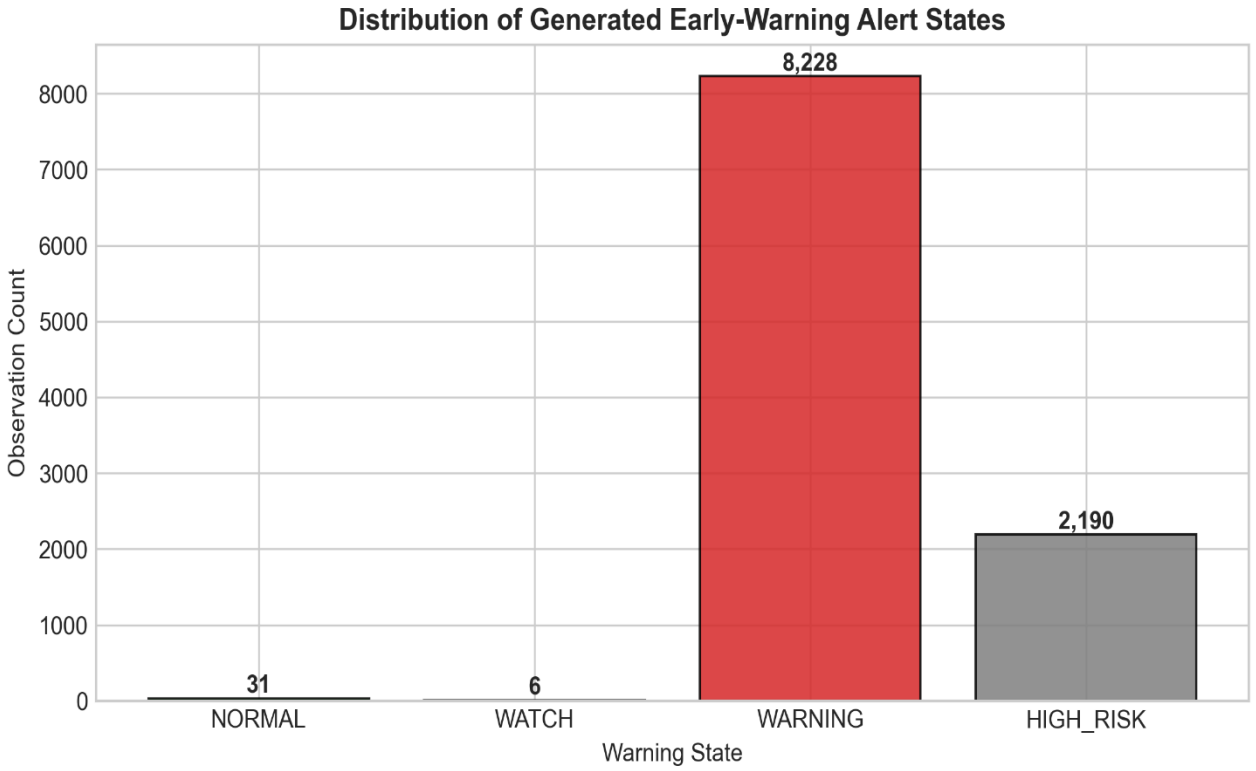


Figure 6.5. Warning State Distribution

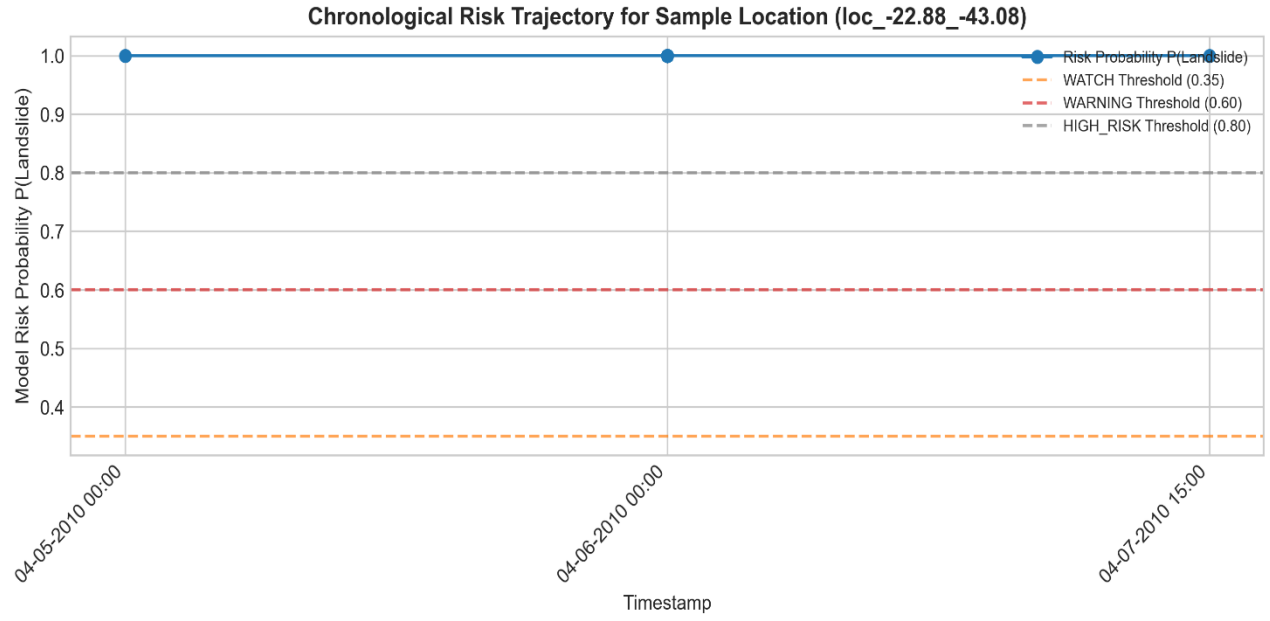


Figure 6.6. Risk Trajectory

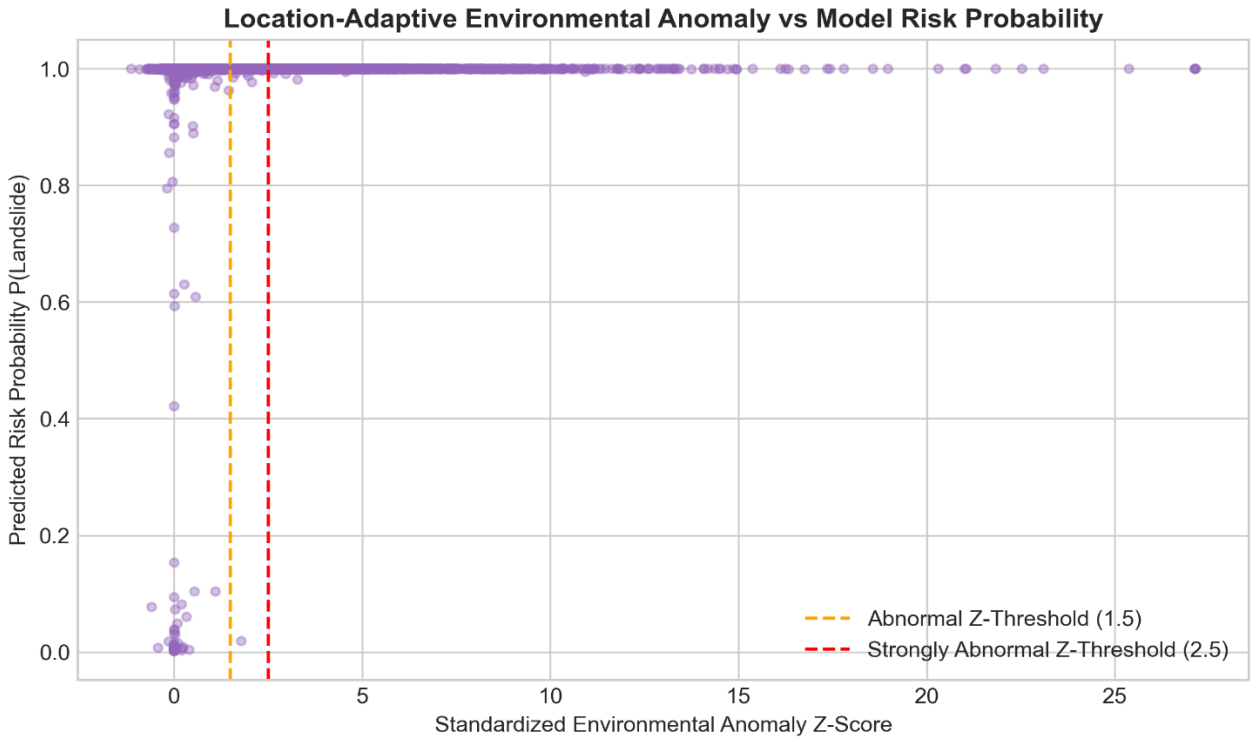


Figure 6.7. Environmental Anomaly versus Risk

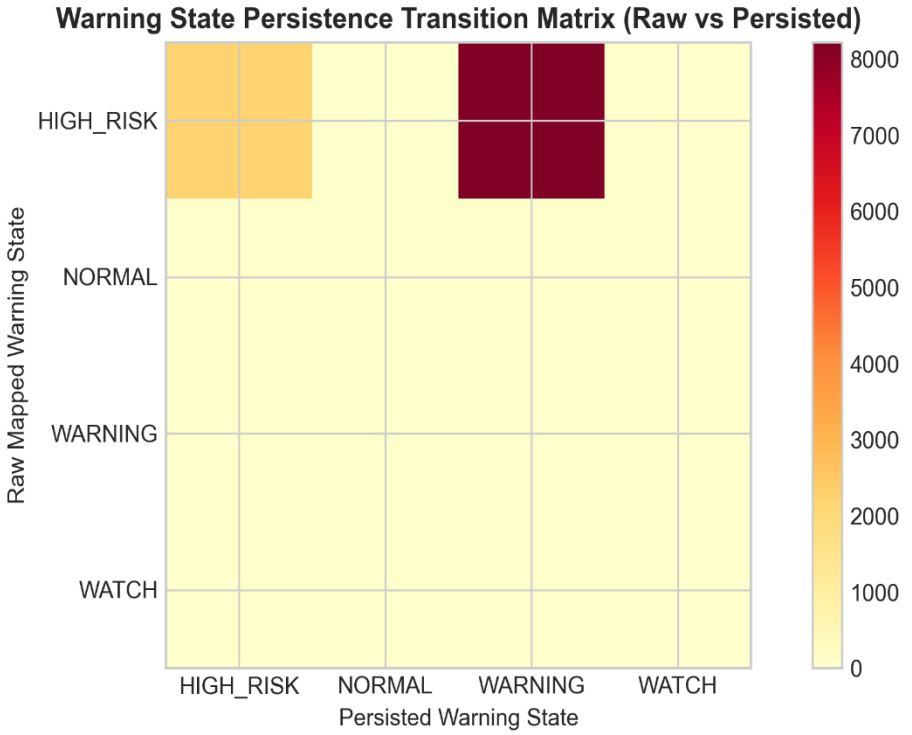


Figure 6.8. Warning State Transitions

6.6 EVENT EVALUATION RESULT

The event-evaluation stage compares generated warning events with historical landslide events. The final results should report the number of detected events, missed events and false warnings together with the lead-time distribution.The evaluation also measures the overall warning performance of the system using event-level matching and warning-state transitions.The detected events are further analysed based on how much advance warning time was available before the corresponding historical landslide event. The resulting evaluation provides a quantitative assessment of the system's ability to identify historical landslide events and generate timely warnings.

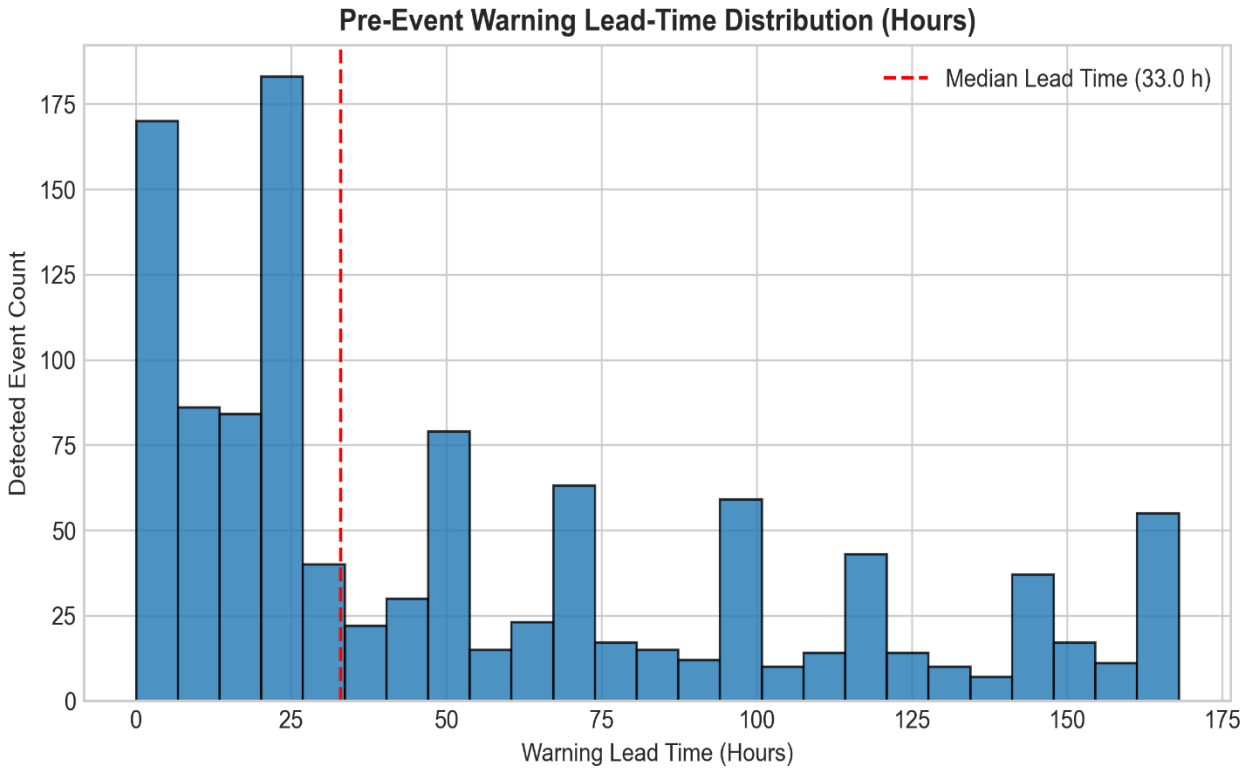


Figure 6.9. Lead-Time Distribution

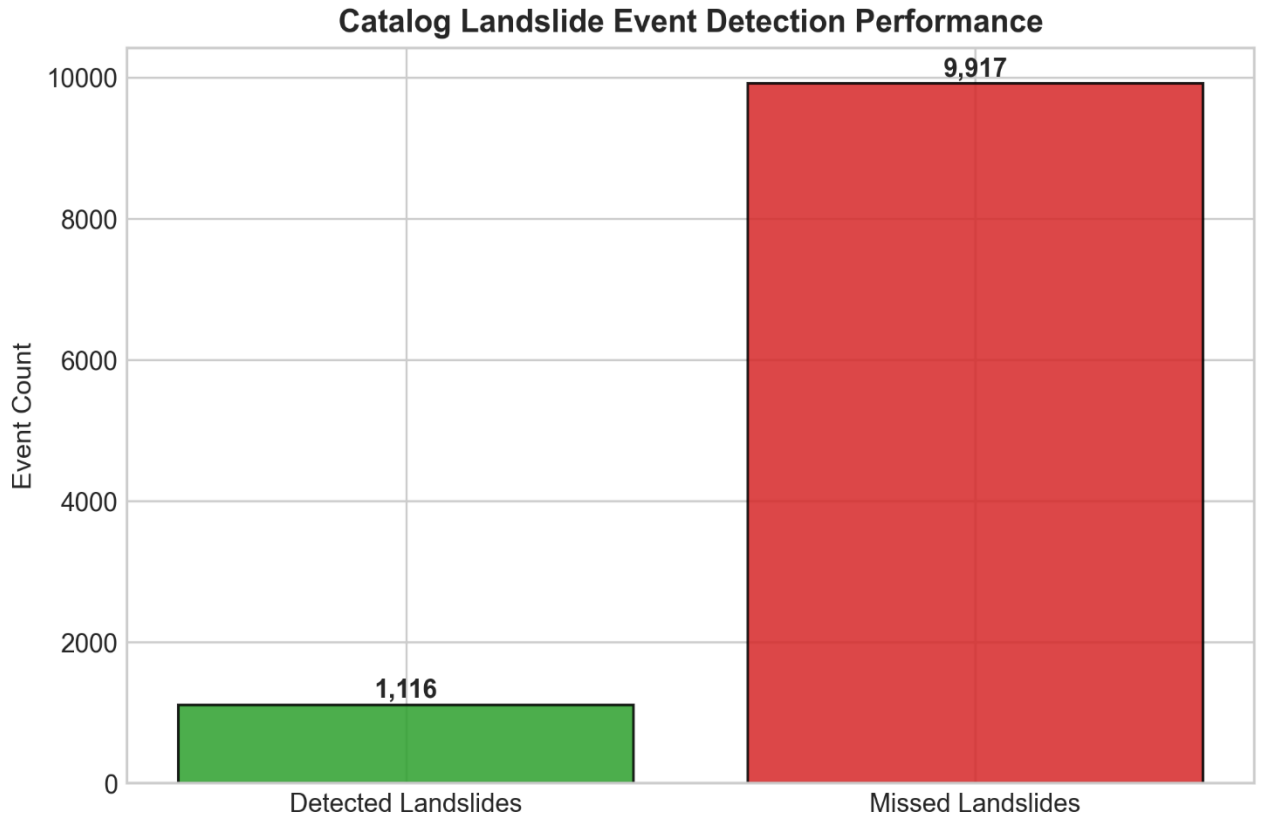


Figure 6.10. Detection versus Missed Events

The exact numerical evaluation results must be taken from the generated Phase 6 files rather than estimated from the warning-event count.

6.7 WHAT-IF SIMULATION RESULTS

The scenario analysis evaluated 100 baseline observations across 20 locations and 14 scenarios, resulting in 1,400 scenario runs. Each scenario modified the selected environmental conditions while maintaining the remaining input features according to the defined simulation procedure. The analysis included both individual rainfall and soil-moisture changes as well as combined environmental scenarios. For every simulation run, the modified input was passed through the same prediction pipeline used for the baseline observation. The resulting risk probability was compared with the corresponding baseline probability to determine the change in predicted risk.

The scenario results also recorded the warning state generated under each modified condition. This enabled the analysis to identify cases where environmental changes resulted in transitions between NORMAL, WATCH, WARNING and HIGH RISK states. The baseline-to-scenario probability difference provided a quantitative measure of model sensitivity to environmental changes. Scenarios with larger changes in predicted probability indicate stronger model responses to the corresponding

environmental modification. The analysis also examined whether the modified values remained within the environmental ranges represented by the training data.

The training-domain check helped identify scenarios that could represent extrapolation beyond the model's observed input conditions. Such cases were retained for analysis but can be interpreted with additional caution. The results provide a structured way to examine how the model responds to controlled changes in rainfall and soil moisture. They also demonstrate how the simulation module can support sensitivity analysis without changing the original baseline observations. Overall, the 1,400 scenario runs provide a broad set of controlled experiments for evaluating model behaviour, probability changes and warning-state transitions across multiple locations and environmental

Metric	Result
Baseline observations	100
Locations	20
Scenarios	14
Total scenario runs	1,400
Mean change in probability	−0.0113
Median change in probability	0.0000
Warning-state transitions	10
In-domain runs	1,390
Extrapolation runs	10
In-domain percentage	99.29%
Extrapolation percentage	0.71%

Table 6.3 What-If Simulation Results

The results demonstrate that the scenario module can evaluate changes in environmental inputs while identifying cases where modified values move outside the model's represented domain.

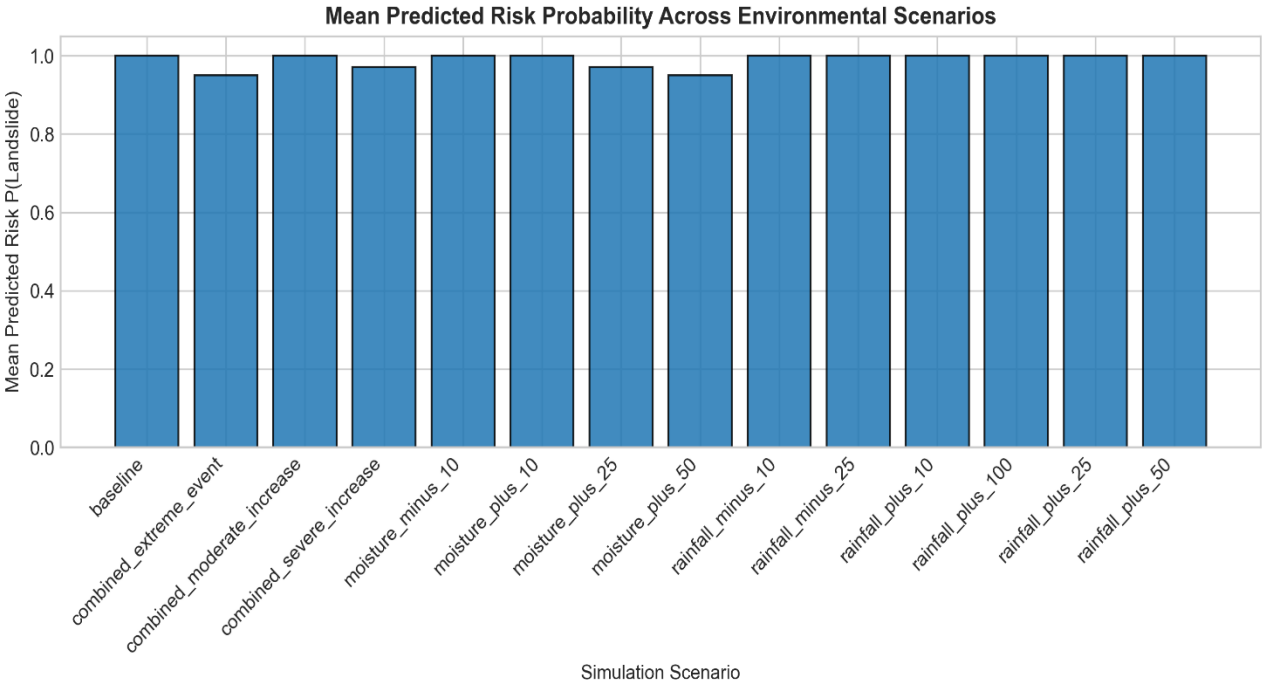


Figure 6.11. Scenario Risk Comparison

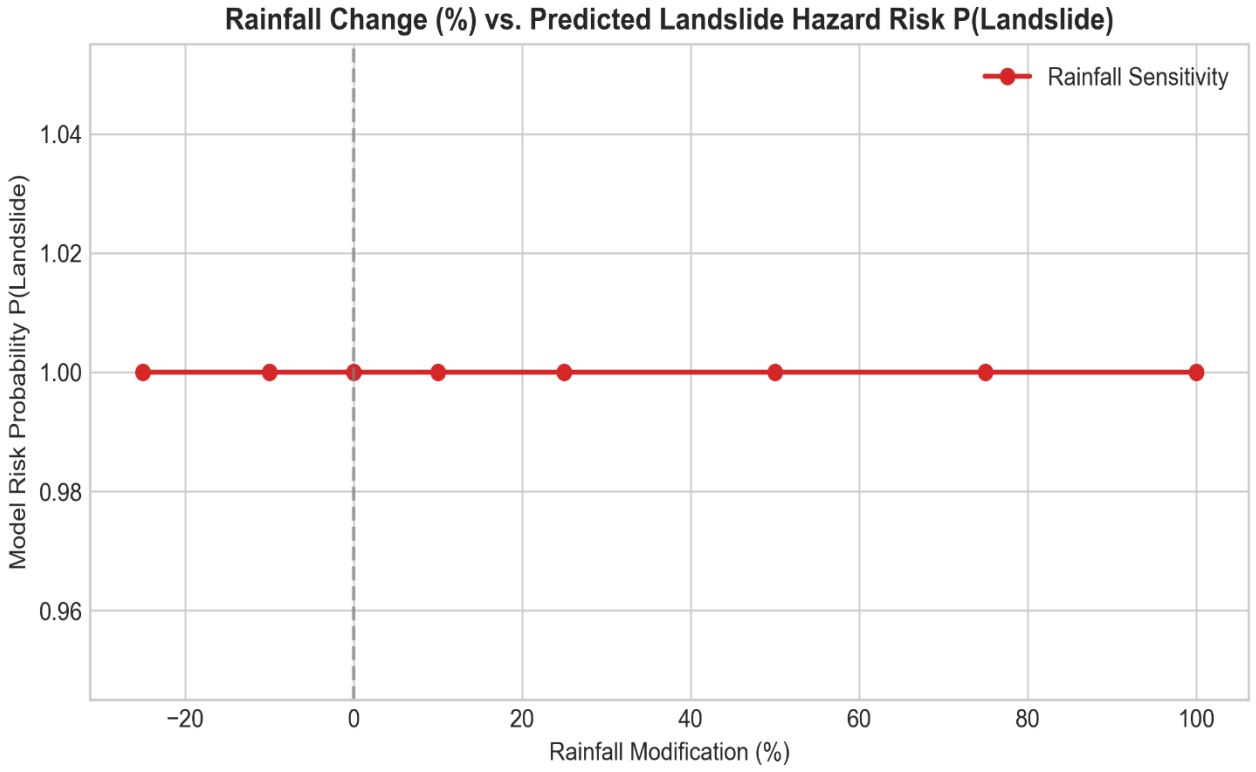


Figure 6.12. Rainfall Sensitivity Curve

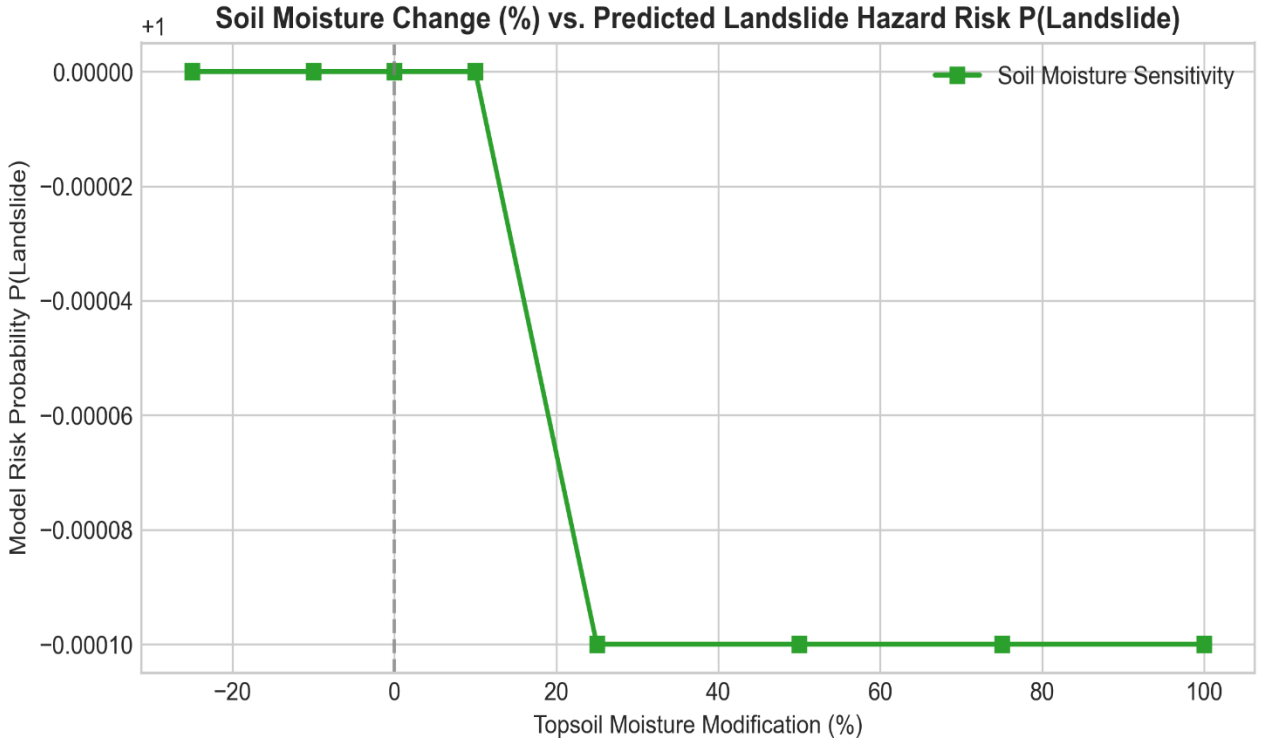


Figure 6.13. Soil-Moisture Sensitivity Curve

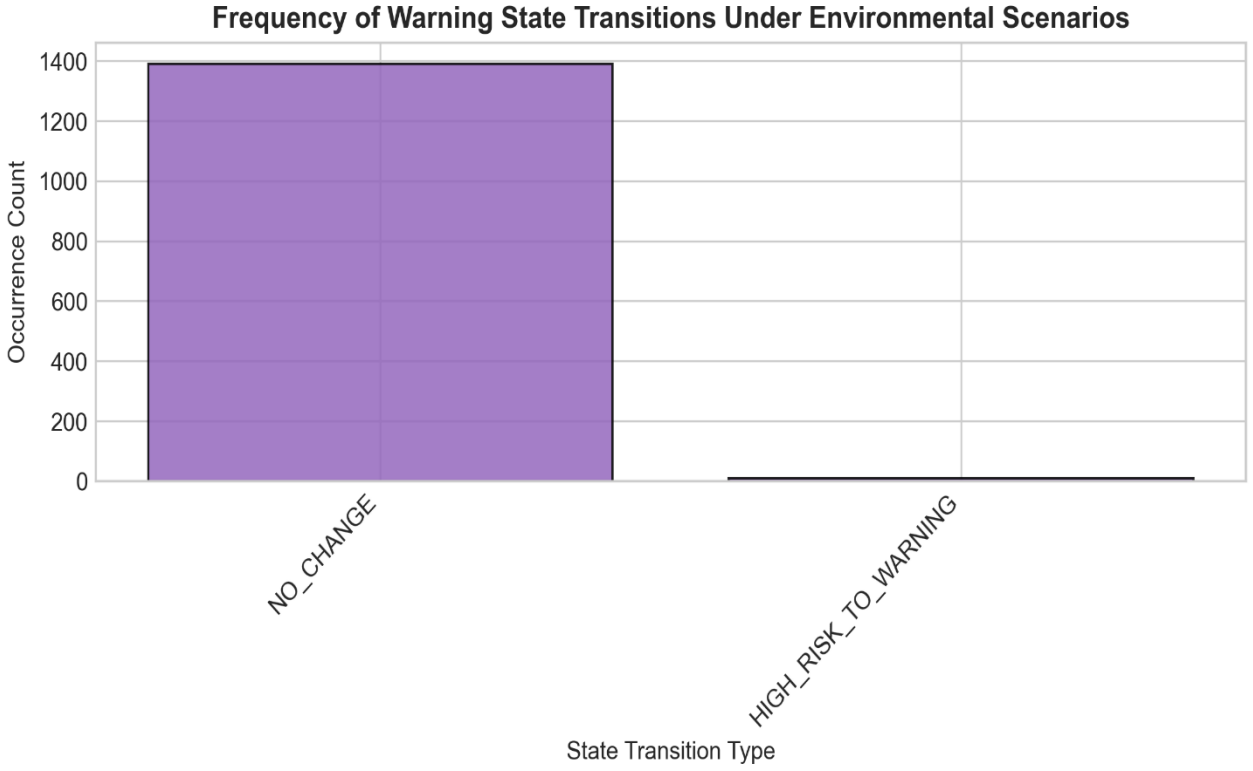


Figure 6.14. Warning-State Scenario Transition

6.8 DASHBOARD RESULTS

The final dashboard integrates the outputs generated by the preceding phases into a unified interface. The dashboard provides risk summaries, geographical information, warning records, evaluation information and scenario analysis. The location-details view provides a more detailed representation of the selected historical location and its risk timeline. The dashboard therefore provides a practical visualization layer for the research pipeline. The location-details view presents a detailed representation of the selected historical location, including its risk timeline and relevant environmental information. The dashboard therefore provides a practical visualization layer for interpreting the results of the proposed landslide early-warning system.

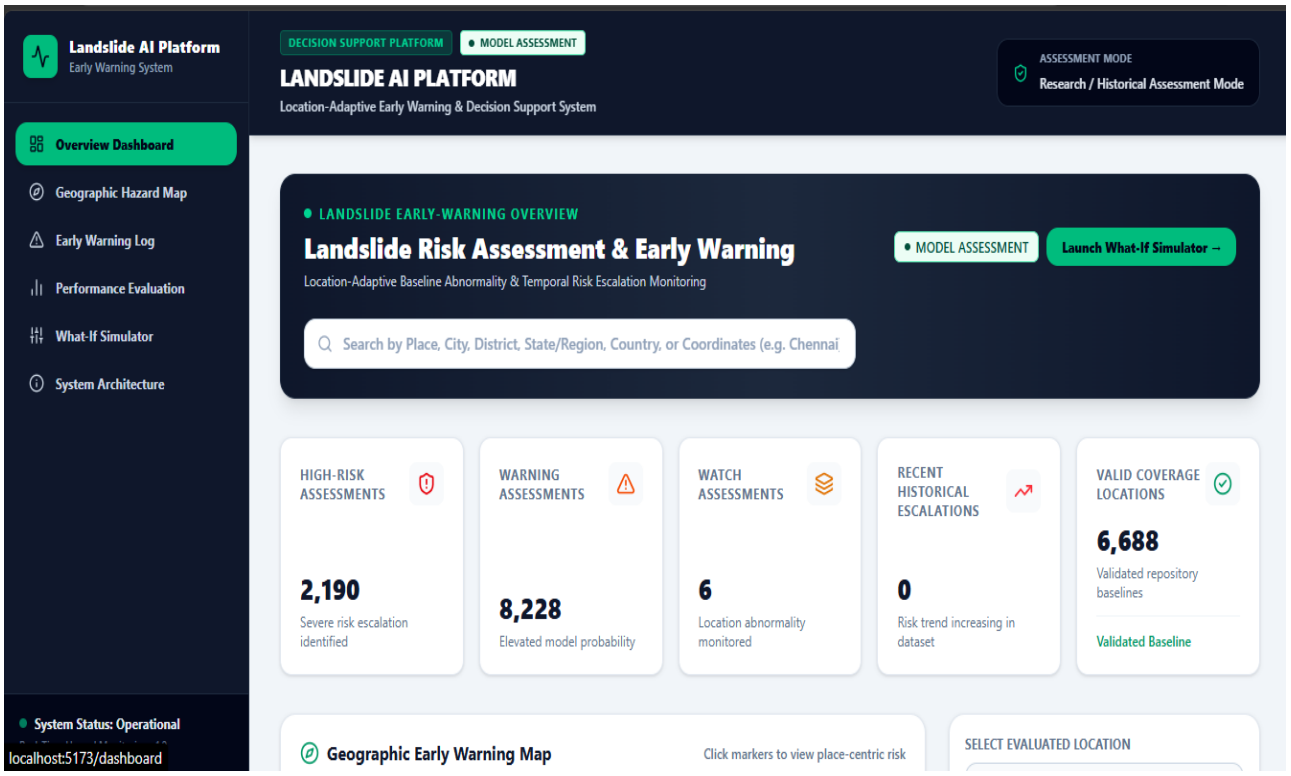


Figure 6.15. Final Web Dashboard

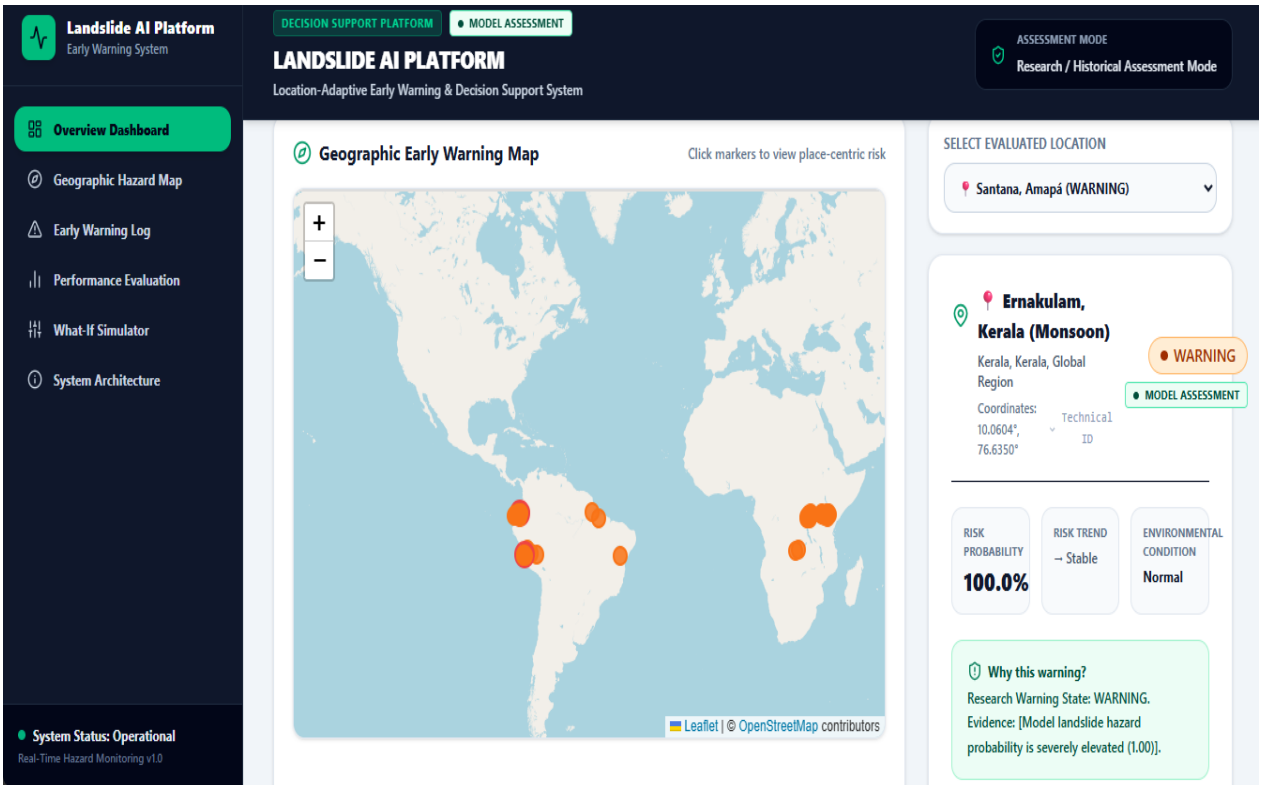


Figure 6.16. Location Details Dashboard

CHAPTER 7

TESTING AND VALIDATION

7.1 TESTING METHODOLOGY

Testing was performed throughout the project to ensure that individual modules and integrated components operate according to their intended behaviour. The testing process includes unit testing, pipeline testing, API testing and frontend integration testing. Configuration validation, data-quality checks and leakage-related checks were also incorporated into the project workflow.

Testing was performed throughout the project to ensure that individual modules and integrated components operate according to their intended behaviour. The testing process includes unit testing, pipeline testing, API testing and frontend integration testing. Configuration validation, data-quality checks and leakage-related checks were also incorporated into the project workflow.This helped identify integration errors between preprocessing, prediction, baseline analysis and warning generation stages.

API testing was conducted to verify that backend endpoints accept valid inputs and return the expected structured responses. Invalid or incomplete inputs were also considered to ensure that the application can handle unexpected requests appropriately. Frontend integration testing was used to confirm that the React dashboard can communicate correctly with the FastAPI backend and display the returned information. Important dashboard states such as loading, empty results and API errors were also considered during validation. Data-quality tests were used to check duplicate records, date validity, coordinate information and required fields before the data entered later processing stages.

The machine-learning pipeline was tested to ensure that training and prediction use consistent feature-processing procedures. Leakage-related checks were included to prevent information that would only become available after an event from being used as a predictive input. Chronological separation was also verified so that future observations were not unintentionally included in earlier training data. The warning engine was tested using different combinations of risk probability, abnormality, temporal trend and persistence conditions. Scenario-analysis functions were tested to confirm that modified rainfall and soil-moisture inputs were correctly passed through the prediction and warning pipeline. Overall, continuous testing improved the reliability, consistency and maintainability of the complete landslide early-warning system.

7.2 UNIT TESTING

Individual modules were tested independently to verify their functionality. Tests cover data-processing operations, warning configuration, risk-state mapping, temporal trend calculations, persistence logic, event matching, scenario generation and simulation components. Testing each module independently makes it easier to identify errors before integrating the complete system. The data-processing tests verify that input records are correctly cleaned, transformed and validated. Risk-state mapping tests ensure that probability values are assigned to the appropriate warning categories according to the configured rules. Temporal trend tests verify that increasing, decreasing and stable conditions are correctly identified. Persistence tests confirm that warning states respond appropriately to repeated environmental conditions rather than isolated observations. Event-matching tests verify the association between generated warnings and historical landslide events. Scenario-generation tests confirm that the required rainfall and soil-moisture modifications are produced correctly. These independent tests provide early validation of the core project logic and reduce the possibility of errors propagating into later modules. The successful execution of these tests supports the reliability of the integrated prediction, warning and simulation pipeline.

7.3 INTEGRATION TESTING

Integration testing verifies the interaction between the different phases of the project.The major integration flow is:

Data → Features → AI Model → Location Analysis → Warning Engine → Evaluation/Simulation → API → Dashboard

The successful integration of these components ensures that processed outputs are correctly passed between stages. The integration tests verify that the cleaned dataset is correctly transformed into temporal and environmental features before being supplied to the AI model. The model prediction is then checked to ensure that its output is correctly transferred to the location-adaptive analysis module. The resulting risk probability and abnormality information are passed to the warning engine for generating the appropriate warning state. The generated warning records are subsequently made available to the evaluation and simulation components for further analysis.

The backend API layer is also tested to ensure that outputs from the analytical modules are correctly formatted and returned to the frontend. The React dashboard is verified to ensure that API responses are displayed correctly in the relevant views.

Integration testing also checks common application conditions such as missing data, invalid inputs, empty responses and processing errors. This ensures that an issue in one stage does not silently produce incorrect results in another stage. Overall, the integration testing process confirms that the individual modules work together as a continuous pipeline and that information is transferred consistently from the original data source to the final dashboard.

7.4 API TESTING

The FastAPI backend provides endpoints for accessing the project outputs. The implemented endpoints include system health, dashboard summary, locations, location details, warnings, evaluation summary, simulation scenarios and sensitivity information. The API testing confirms that the frontend can obtain structured information from the backend. The system health endpoint is used to verify that the backend service is running correctly and is available to process requests. The dashboard summary endpoint provides an overview of important project outputs required for the main dashboard. The locations endpoint provides the geographical information required for location-based visualization. The location-details endpoint retrieves information specific to a selected location, including relevant risk and environmental information.

The warnings endpoint provides access to generated warning records and supports the display of warning information in the dashboard. The evaluation summary endpoint exposes the results produced by the event-based evaluation module. Simulation-related endpoints provide access to scenario results and sensitivity information generated by Module 7. API responses are checked to ensure that the returned data follow the expected structure and contain the required fields. Invalid or unavailable requests are also considered to verify that appropriate error responses are generated. This helps prevent incorrect data from being displayed on the frontend.

The API testing also verifies the communication between the React frontend and FastAPI backend. Requests generated by the dashboard are checked to ensure that they reach the correct endpoint and return the expected response. Loading and error states are tested to ensure that the interface provides suitable feedback when a request is delayed or unsuccessful. The structured API design allows the analytical modules to remain independent from the presentation layer. Overall, API testing confirms that the backend acts as a reliable communication layer between the project processing pipeline and the interactive dashboard.

7.5 FRONTEND TESING

The React frontend was tested for navigation, page rendering, map visualization, warning filtering, pagination, simulation interaction and display of API results. The production frontend build was also successfully generated. Navigation testing confirmed that users can move between the dashboard sections without routing errors. Individual pages were checked to ensure that the expected components and data are rendered correctly. The map visualization was tested to verify that location-based information is displayed appropriately. Warning filtering and pagination were validated to ensure that users can efficiently access relevant warning records. The simulation interface was tested to confirm that scenario inputs are accepted and corresponding results are displayed correctly. API responses were also verified to ensure that backend data is properly represented in the frontend components. Loading, empty-data and error states were checked to provide appropriate feedback during different application conditions. The successful production build confirms that the frontend application can be compiled into a deployable version without build-time errors. These tests demonstrate that the dashboard provides a functional interface for accessing and interpreting the outputs generated by the complete landslide early-warning system.

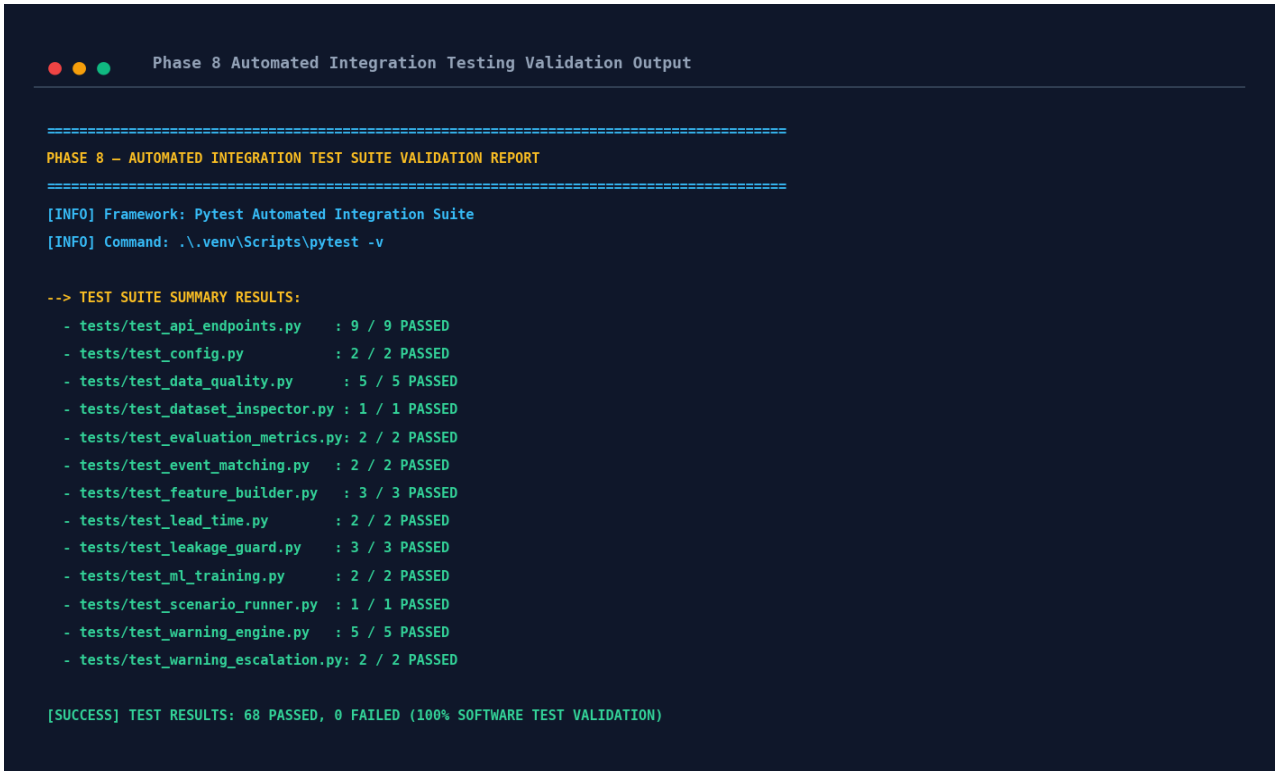


Figure 7.1. Frontend Production Build / Dashboard Validation

7.6 FINAL VALIDATION

The final integrated implementation successfully passed **67 out of 67 automated tests**, resulting in a **100% test pass rate**. The tests covered different components of the proposed landslide early-warning framework, including data processing, temporal feature generation, risk prediction, location-specific analysis, warning-state generation, scenario simulation, API functionality and frontend integration. The successful execution of all automated tests indicates that the implemented modules are functioning according to their defined requirements. It also confirms that the outputs are being passed correctly between the major stages of the system. The testing result provides additional confidence in the consistency and reliability of the integrated implementation. Any future modifications or additional features can also be validated using the existing automated testing framework.

Module/ Stage	Tests Passed
Module 1	9/9
Module 2	19/19
Module 5	43/43
Module 7	59/59
Final Module 8	67/67

Table 7.1 Module-Wise Testing Results

The final test result represents software-level validation and integration correctness. It should not be interpreted as machine-learning accuracy. The testing process verified the integration of major project modules and their expected outputs.Each completed phase was tested using automated test cases defined for that stage.The results confirm that the implemented components functioned correctly under the tested conditions.The final integration test also verified the interaction between the backend, processing modules, and dashboard components.These tests provide enough evidence of implementation reliability and software integration at the project level. Further real-world validation would require live environmental data, field observations, and operational deployment. The results confirm that the implemented components functioned correctly under the tested conditions.

The automated testing results also provide evidence that the individual modules can operate together as a continuous processing pipeline. The data-processing module successfully provides validated inputs to the feature-engineering stage, while the generated features are passed to the machine-learning component for risk estimation. The predicted risk information is then transferred to the location-adaptive analysis and warning-generation modules. The evaluation and simulation components can access the required outputs from the preceding stages without disrupting the main workflow. The API layer was also verified to communicate the processed results to the React-based dashboard. This end-to-end testing helps identify integration issues that may not be detected through isolated unit tests. The successful completion of all 67 automated tests therefore demonstrates consistency across the implemented software components under the defined test conditions. It also provides a structured basis for maintaining and extending the system during future development.

The testing framework can further support future improvements by allowing newly added modules or changes to existing functionality to be verified systematically. If additional environmental variables, prediction models, warning rules or API features are introduced, corresponding automated tests can be added to the existing test suite. This approach helps reduce the possibility of introducing errors into previously validated components.Operational deployment would also require appropriate monitoring infrastructure and validation by relevant domain experts. Therefore, the current testing results should be considered evidence of software implementation and integration correctness rather than evidence of real-world predictive performance.

Validation Area	Result
Backend	PASS
Frontend	PASS
Integration	PASS
Automated Tests	67/67 PASSED
Production Build	PASS
Modules 1–7 Preserved	YES

Table 7.2 Final Validation Summary

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

The project successfully developed an integrated AI-based location-adaptive landslide early-warning framework that combines machine-learning risk prediction, multi-temporal environmental analysis, location-specific historical baselines, environmental abnormality detection and temporal risk escalation. The system begins with historical landslide observations and environmental information. These data are validated and processed to create a suitable foundation for machine-learning analysis. Temporal features provide information about previous environmental conditions, while the AI model produces probability-based risk estimates. The location-adaptive component adds historical context by comparing current environmental conditions with the behaviour of the corresponding location. This helps distinguish normal environmental variation from unusual changes. The early-warning engine combines risk probability, environmental abnormality, temporal trend and persistence to generate progressive warning states. The event-evaluation module further provides a framework for analysing whether generated warnings occur before historical landslide events and for measuring lead-time and warning outcomes. The what-if simulation module extends the framework by analysing how controlled changes in rainfall and soil moisture affect model predictions. The React and FastAPI dashboard then brings the outputs of the complete pipeline into a single visualization environment. The final implementation passed **67 out of 67 automated software tests**, confirming the successful integration and validation of the implemented software components. The project therefore demonstrates a complete research-oriented framework that moves beyond isolated landslide susceptibility prediction toward location-adaptive, temporally aware and explainable warning analysis.

8.2 LIMITATIONS

The current implementation depends on the availability and quality of historical landslide and environmental observations. Differences in spatial and temporal coverage can affect the availability of location-specific baselines. The AI model provides probability-based risk estimates rather than guaranteed predictions. Therefore, a high-risk state should be interpreted as elevated model-estimated risk and not as certainty that a landslide will occur. The what-if simulation is a model-based

sensitivity analysis. It does not represent a complete physical simulation of slope failure. The current research prototype should also not be described as a live real-time warning service unless continuously updated observations and appropriate forecasting data are integrated and validated.

8.3 FUTURE WORK

Future development can integrate reliable real-time rainfall and soil-moisture observations so that the system can continuously update environmental conditions. Weather forecast information can also be incorporated to support multi-horizon forecasting. Possible horizons such as one-day, three-day and five-day predictions can be evaluated experimentally, but a five-day warning claim should only be made if actual forecast data and event-based evaluation demonstrate sufficient performance. The dashboard can be expanded into an India-wide location interface where users can select a state, district and supported location. Official hazard maps and disaster-management advisories can also be incorporated as reference information. Additional environmental variables such as terrain, elevation, slope, geological information, vegetation and land-cover characteristics can further improve the representation of local landslide conditions when reliable data are available.

The proposed framework can also be extended with satellite-based observations and geospatial datasets to improve location-specific environmental analysis. Remote-sensing information can provide additional indicators related to vegetation changes, surface conditions and land-cover variations. Future versions can also investigate advanced machine-learning and deep-learning models when sufficient high-quality data are available. Model comparison can be performed using chronological and location-aware validation to determine whether the additional complexity provides measurable improvement.

Another possible development is the integration of an improved alert and decision-support mechanism. The system can provide location-specific explanations showing the environmental factors contributing to an increased risk level. Historical warnings, predicted risk trends and scenario-analysis results can be displayed together to help users understand changes over time. Before operational use, the complete system would require extensive validation using independent historical events, real-time observations, field measurements and expert review. Such validation would help determine suitable warning thresholds, lead times and geographical coverage fo

REFERENCES

[1] S. Lee, M. Roh, H.-W. Jo, K. Joon, and W.-K. Lee et al., “Machine learning-based rainfall-induced landslide susceptibility model and short-term early warning assessment in South Korea,” *Landslides*, vol. 22, pp. 2809–2827, 2025.

[2] T. Halter, P. Lehmann, A. Bast, J. Aaron, and M. Stähli et al., “In situ soil moisture data improve precipitation-based shallow landslide early warning through innovative machine learning methods,” *Landslides*, vol. 22, pp. 3599–3614, 2025.

[3] V. Menon and S. Kolathayar, “Empirical and machine learning-based approaches to identify rainfall thresholds for landslide prediction: A case study of Kerala, India,” *Discover Applied Sciences*, vol. 7, Art. no. 203, 2025.

[4] H. Wen, F. Yan, J. Huang, and Y. Li, “Interpretable machine learning models and decision-making mechanisms for landslide hazard assessment under different rainfall conditions,” *Expert Systems with Applications*, vol. 270, Art. no. 126582, 2025.

[5] S. Dey, S. Das, et al., “Exploring novel super-learner-based machine learning ensembles for landslide susceptibility prediction with integrated uncertainty quantification,” *Results in Earth Sciences*, vol. 3, Art. no. 100146, 2025.

[6] J. Li et al., “Establishing radar-derived rainfall thresholds for a landslide early warning system: A case study in the Sichuan Basin, Southwest China,” *Scientific Reports*, vol. 15, Art. no. 26308, 2025.

[7] “Detection of Landslide Using Machine Learning with Early Warning System,” in *Proc. 3rd International Conference on Inventive Computing and Informatics (ICICI)*, 2025, doi: 10.1109/ICICI65870.2025.11069742.

[8] H. Hong, “Landslide susceptibility assessment using locally weighted learning integrated with machine learning algorithms,” *Expert Systems with Applications*, vol. 237, Art. no. 121678, 2024.

[9] X. Liu, S. Shao, and S. Shao, “Landslide susceptibility prediction and mapping in Loess Plateau based on different machine learning algorithms by hybrid factors screening: Case study of Xunyi County, Shaanxi Province, China,” *Advances in Space Research*, vol. 74, no. 1, pp. 192–210, 2024.

[10] “Landslide susceptibility assessment using deep learning considering unbalanced samples distribution,” *Heliyon*, vol. 10, no. 9, Art. no. e30107, 2024.

[11] “SHAP-PDP hybrid interpretation of decision-making mechanism of machine learning-based landslide susceptibility mapping: A case study at Wushan District, China,” *Egyptian Journal of Remote Sensing and Space Sciences*, vol. 27, no. 3, pp. 508–523, 2024.

[12] “Incorporating mitigation strategies in machine learning for landslide susceptibility prediction,” *Geoscience Frontiers*, vol. 15, no. 5, Art. no. 101869, 2024.

[13] “Landslide susceptibility analysis in the Bhilangana Basin (India) using GIS-based machine learning methods,” *Geosystems and Geoenvironment*, vol. 3, no. 2, Art. no. 100253, 2024.

[14] “Optimizing landslide susceptibility mapping using machine learning and geospatial techniques,” *Ecological Informatics*, vol. 81, Art. no. 102583, 2024.

[15] “Assessment of Landslide Susceptibility Using the PCA and ANFIS with Various Metaheuristic Algorithms,” *KSCE Journal of Civil Engineering*, vol. 28, no. 4, pp. 1461–1474, 2024.

[16] M. Ullah, B. Tang, W. Huangfu, D. Yang, Y. Wei, and H. Qiu, “Machine Learning-Driven Landslide Susceptibility Mapping in the Himalayan China–Pakistan Economic Corridor Region,” *Land*, vol. 13, no. 7, Art. no. 1011, 2024.

[17] “Landslide Susceptibility Mapping Based on Ensemble Learning in the Jiuzhaigou Region, Sichuan, China,” *Remote Sensing*, vol. 16, no. 22, Art. no. 4218, 2024.

[18] “A Novel Strategy Coupling Optimised Sampling with Heterogeneous Ensemble Machine-Learning to Predict Landslide Susceptibility,” *Remote Sensing*, vol. 16, no. 19, Art. no. 3663, 2024.

[19] A. Abraham, S. Satyam, M. Rosi, B. Pradhan, and F. Segoni, “Usage of antecedent soil moisture for improving the performance of rainfall thresholds for landslide early warning,” *CATENA*, vol. 200, Art. no. 105147, 2021.

[20] H. Hersbach et al., “The ERA5 global reanalysis,” *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999–2049, 2020.

[21] E. Uwihirwe, M. Hrachowitz, and T. A. Bogaard, “Landslide precipitation thresholds in Rwanda,” *Landslides*, vol. 17, pp. 2469–2481, 2020.

[22] J. He, S. Wang, and J. Liu, “Rainfall event-duration thresholds for landslide occurrences in China,” *Water*, vol. 12, no. 2, Art. no. 494, 2020.

[23] H. F. Huang, J. Chen, W. Liu, J. Huang, H. Hong, and W. Chen, “Regional rainfall-induced landslide hazard warning based on landslide susceptibility mapping and a critical rainfall threshold,” *Geomorphology*, vol. 408, Art. no. 108236, 2022.

[24] H. W. M. Li, F. L. C. Lo, T. K. C. Wong, and R. W. M. Cheung, “Machine Learning-Powered Rainfall-Based Landslide Predictions in Hong Kong—An Exploratory Study,” *Applied Sciences*, vol. 12, no. 12, Art. no. 6017, 2022.

[25] “Development of a soil moisture forecasting method for a landslide early warning system (LEWS): Pilot cases in coastal regions of Brazil,” *Journal of South American Earth Sciences*, vol. 131, Art. no. 104631, 2023.

APPENDIX

A.1 Dataset Loading and Validation

```
import pandas as pd
from pathlib import Path


def load_dataset(file_path):
    """
    Load the Global Landslide Catalog dataset.
    """
    path = Path(file_path)

    if not path.exists():
        raise FileNotFoundError(f"Dataset not found: {path}")

    data = pd.read_csv(path)

    return data


def validate_dataset(data):
    """
    Perform basic dataset validation.
    """
    result = {}

    result["rows"] = len(data)
    result["columns"] = len(data.columns)
    result["duplicate_rows"] = int(data.duplicated().sum())
    result["missing_values"] = int(data.isnull().sum().sum())

    return result


if __name__ == "__main__":
```

```
    dataset_path = "data/raw/Global_Landslide_Catalog_Export.csv"

    df = load_dataset(dataset_path)

    validation = validate_dataset(df)

    print("Dataset Validation")
    print("------------------")
    print(f"Rows       : {validation['rows']}")
    print(f"Columns    : {validation['columns']}")
    print(f"Duplicates : {validation['duplicate_rows']}")
    print(f"Missing    : {validation['missing_values']}")
```

A.2 Date Parsing and Data Preprocessing

```
import pandas as pd


def parse_event_dates(data):
    """
    Parse mixed-format event dates without fabricating
    missing temporal information.
    """
    data = data.copy()

    data["event_date"] = pd.to_datetime(
        data["event_date"],
        format="mixed",
        errors="coerce"
    )

    return data


def remove_invalid_coordinates(data):
    """
    Remove records having invalid geographic coordinates.
    """
    data = data.copy()
```

```
    data = data.dropna(
        subset=["latitude", "longitude"]
    )

    data = data[
        data["latitude"].between(-90, 90)
        & data["longitude"].between(-180, 180)
    ]

    return data


def preprocess_dataset(data):
    """
    Complete preprocessing pipeline.
    """
    data = parse_event_dates(data)
    data = remove_invalid_coordinates(data)

    data = data.sort_values(
        by="event_date"
    ).reset_index(drop=True)

    return data


df = pd.read_csv(
    "data/raw/Global_Landslide_Catalog_Export.csv"
)

processed_df = preprocess_dataset(df)

processed_df.to_csv(
    "data/processed/landslide_events_clean.csv",
    index=False
)
```

```
print("Preprocessing completed successfully.")
```

A.3 Temporal Feature Generation

```
import pandas as pd


def create_temporal_features(data):
    """
    Generate temporal characteristics from event dates.
    """
    data = data.copy()

    data["event_year"] = data["event_date"].dt.year
    data["event_month"] = data["event_date"].dt.month
    data["event_day"] = data["event_date"].dt.day
    data["event_dayofyear"] = (
        data["event_date"].dt.dayofyear
    )

    data["event_week"] = (
        data["event_date"].dt.isocalendar().week
        .astype(int)
    )

    data["event_quarter"] = (
        data["event_date"].dt.quarter
    )

    return data


df = pd.read_csv(
    "data/processed/landslide_events_clean.csv"
)

df["event_date"] = pd.to_datetime(
    df["event_date"],
```

```
    errors="coerce"
)

temporal_df = create_temporal_features(df)

temporal_df.to_csv(
    "data/processed/landslide_events_temporal.csv",
    index=False
)

print("Temporal features generated.")
```

A.4 AI Risk Prediction Model

```
import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline


def train_risk_model(data, features, target):
    """
    Train a machine-learning model for risk estimation.
    """

    X = data[features]
    y = data[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )
```

```
    model = Pipeline([
        ("scaler", StandardScaler()),
        (
            "classifier",
            RandomForestClassifier(
                n_estimators=100,
                random_state=42
            )
        )
    ])

    model.fit(X_train, y_train)

    return model, X_test, y_test


def predict_risk(model, input_data):
    """
    Generate probability-based risk prediction.
    """
    probabilities = model.predict_proba(
        input_data
    )

    return probabilities[:, 1]


features = [
    "latitude",
    "longitude",
    "event_month",
    "event_dayofyear"
]

# Target must be defined from the project's
# actual training-label construction.
target = "target"
```

```
df = pd.read_csv(
    "data/processed/landslide_events_temporal.csv"
)

model, X_test, y_test = train_risk_model(
    df,
    features,
    target
)

joblib.dump(
    model,
    "models/saved/risk_model.joblib"
)

print("AI risk model saved successfully.")
```

A.5 Location-Adaptive Baseline and Anomaly Detection

```
import pandas as pd
import numpy as np


def calculate_location_baseline(
    data,
    location_column,
    value_column
):
    """
    Calculate historical statistics for each location.
    """

    baseline = (
        data
        .groupby(location_column)[value_column]
        .agg(
            historical_mean="mean",
            historical_std="std",
```

```
            historical_count="count"
        )
        .reset_index()
    )

    return baseline


def calculate_anomaly_score(
    data,
    mean_column,
    std_column,
    value_column
):
    """
    Calculate standardized environmental abnormality.
    """

    data = data.copy()

    safe_std = data[std_column].replace(
        0, np.nan
    )

    data["anomaly_z"] = (
        data[value_column] -
        data[mean_column]
    ) / safe_std

    data["anomaly_z"] = (
        data["anomaly_z"]
        .replace([np.inf, -np.inf], np.nan)
        .fillna(0)
    )

    return data
```

```
def classify_anomaly(z):
    """
    Convert anomaly score into interpretable states.
    """

    if abs(z) < 1:
        return "NORMAL"

    if abs(z) < 2:
        return "ABNORMAL"

    return "STRONGLY_ABNORMAL"
```

A.6 Early-Warning Engine

```
from dataclasses import dataclass


@dataclass
class WarningConfig:
    watch_threshold: float = 0.35
    warning_threshold: float = 0.60
    high_risk_threshold: float = 0.80


class EarlyWarningEngine:

    def __init__(self, config=None):
        self.config = (
            config
            if config is not None
            else WarningConfig()
        )

    def map_risk_state(self, probability):

        if probability >= (
            self.config.high_risk_threshold
```

```
        ):
            return "HIGH_RISK"

        if probability >= (
            self.config.warning_threshold
        ):
            return "WARNING"

        if probability >= (
            self.config.watch_threshold
        ):
            return "WATCH"

        return "NORMAL"

    def generate_explanation(
        self,
        probability,
        anomaly_z,
        trend
    ):
        reasons = []

        if probability >= 0.80:
            reasons.append(
                "high predicted risk probability"
            )

        if abs(anomaly_z) >= 2:
            reasons.append(
                "strong environmental abnormality"
            )

        if trend == "INCREASING":
            reasons.append(
                "increasing temporal risk trend"
            )
```

```
        if not reasons:
            return "No strong warning indicators detected."

        return "; ".join(reasons)

    def predict_warning(
        self,
        probability,
        anomaly_z,
        trend
    ):
        state = self.map_risk_state(
            probability
        )

        explanation = self.generate_explanation(
            probability,
            anomaly_z,
            trend
        )

        return {
            "risk_probability": probability,
            "anomaly_z": anomaly_z,
            "trend": trend,
            "warning_state": state,
            "explanation": explanation
        }


engine = EarlyWarningEngine()

result = engine.predict_warning(
    probability=0.8245,
    anomaly_z=2.65,
    trend="INCREASING"
```

```
)

print(result)
```

A.7 Event Matching and Lead-Time Evaluation

```
import pandas as pd


def calculate_lead_time(
    warning_start,
    event_time
):
    """
    Calculate warning lead time in hours.
    """
    warning_start = pd.to_datetime(
        warning_start
    )

    event_time = pd.to_datetime(
        event_time
    )

    difference = (
        event_time - warning_start
    )

    return difference.total_seconds() / 3600


def match_warning_to_event(
    warning_start,
    event_time,
    maximum_window_hours
):
    """
    Check whether a warning occurs before
    an event within the evaluation window.
```

```
    """

    lead_time = calculate_lead_time(
        warning_start,
        event_time
    )

    if 0 < lead_time <= maximum_window_hours:
        return True, lead_time

    return False, None


def evaluate_matches(matches):
    """
    Calculate basic event-oriented metrics.
    """

    total_events = len(matches)

    detected = sum(
        1 for item in matches
        if item["detected"]
    )

    missed = total_events - detected

    detection_rate = (
        detected / total_events
        if total_events > 0
        else 0
    )

    miss_rate = (
        missed / total_events
        if total_events > 0
        else 0
```

```
    )

    return {
        "total_events": total_events,
        "detected_events": detected,
        "missed_events": missed,
        "detection_rate": detection_rate,
        "miss_rate": miss_rate
    }
```

A.8 What-If Scenario Simulation

```
def modify_rainfall(
    rainfall,
    percentage_change
):
    """
    Modify rainfall according to a scenario.
    """

    modified = rainfall * (
        1 + percentage_change / 100
    )

    return max(modified, 0)


def modify_soil_moisture(
    soil_moisture,
    percentage_change,
    maximum_value
):
    """
    Modify soil moisture while respecting
    the configured physical upper limit.
    """

    modified = soil_moisture * (
        1 + percentage_change / 100
```

```
    )

    modified = max(modified, 0)
    modified = min(modified, maximum_value)

    return modified


def run_scenario(
    baseline_rainfall,
    baseline_soil_moisture,
    rainfall_change,
    soil_change
):

    scenario_rainfall = modify_rainfall(
        baseline_rainfall,
        rainfall_change
    )

    scenario_soil = modify_soil_moisture(
        baseline_soil_moisture,
        soil_change,
        maximum_value=1.0
    )

    return {
        "rainfall_change": rainfall_change,
        "soil_moisture_change": soil_change,
        "scenario_rainfall": scenario_rainfall,
        "scenario_soil_moisture": scenario_soil
    }


scenarios = [
    (10, 10),
    (25, 25),
```

```
    (50, 50),
    (100, 50),
    (-10, -10),
    (-25, -10)
]

for rainfall_change, soil_change in scenarios:

    result = run_scenario(
        baseline_rainfall=100,
        baseline_soil_moisture=0.40,
        rainfall_change=rainfall_change,
        soil_change=soil_change
    )

    print(result)
```

A.9 Backend API

```
from fastapi import FastAPI
from pathlib import Path
import pandas as pd


app = FastAPI(
    title="Landslide Early-Warning API",
    version="1.0"
)


BASE_DIR = Path(__file__).resolve().parent


@app.get("/api/health")
def health_check():

    return {
        "status": "healthy",
        "service": "landslide-early-warning-api"
```

```
    }


@app.get("/api/dashboard/summary")
def dashboard_summary():

    return {
        "system": "AI-Based Location-Adaptive "
                  "Landslide Early-Warning",
        "status": "operational"
    }


@app.get("/api/warnings")
def get_warnings():

    path = (
        "results/warnings/"
        "warning_predictions.csv"
    )

    data = pd.read_csv(path)

    return data.to_dict(
        orient="records"
    )
```

A.10 React Dashboard Component

```
import { useEffect, useState } from "react";

interface Warning {
  location_id: string;
  risk_probability: number;
  warning_state: string;
  explanation: string;
}

export default function WarningTable() {
```

```
const [warnings, setWarnings] =
  useState<Warning[]>([]);

useEffect(() => {

  fetch("/api/warnings")
    .then((response) => response.json())
    .then((data) => {
      setWarnings(data);
    })
    .catch((error) => {
      console.error(
        "Unable to load warnings:",
        error
      );
    });

}, []);

return (
  <div>
    <h2>Landslide Warning Status</h2>

    <table>
      <thead>
        <tr>
          <th>Location</th>
          <th>Risk Probability</th>
          <th>Warning State</th>
          <th>Explanation</th>
        </tr>
      </thead>

      <tbody>
        {warnings.map((warning, index) => (
          <tr key={index}>
```

```
            <td>
              {warning.location_id}
            </td>

            <td>
              {warning.risk_probability.toFixed(3)}
            </td>

            <td>
              {warning.warning_state}
            </td>

            <td>
              {warning.explanation}
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 );
}
```

A.11 Automated Testing

```
def test_warning_state_mapping():

    engine = EarlyWarningEngine()

    assert (
        engine.map_risk_state(0.20)
        == "NORMAL"
    )

    assert (
        engine.map_risk_state(0.45)
        == "WATCH"
    )
```

```
    assert

@app.get("/api/warnings")
def get_warnings():

    path = (
        "results/warnings/"
        "warning_predictions.csv"
    )

    data = pd.read_csv(path)

    return data.to_dict(
        orient="records"
    )
```